

운영체제 4

남궁명수

[1-1] 병행성: 개요	3
[1] 예제: 쓰레드 생성	5
[2] 훨씬 더 어려운 이유: 데이터 공유	9
[3] 제어 없는 스케줄링	12
[3] 원자성에 대한 바람	14
[4] 또 다른 문제: 상대 기다리기	16
[6] 정리: 왜 운영체제에서?	17
[1-2] 병행성: 쓰레드 API	18
[1] 쓰레드 생성	18
[2] 쓰레드 종료	20
[3] 락(Lock)	24
[4] 컨디션 변수(Condition Variable)	25
[5] 컴파일과 실행	27
[6] 요약	27
[1-3] 병행성: 락	29
[1] 락: 기본 개념	29
[2] Pthread 락	30
[3] 락 구현	31
[4] 락의 평가	32
[5] 인터럽트 제어	33
[6] Test-And-Set(Atomic Exchange)	35
[7] 진짜 돌아가는 스핀 락의 구현	37
[8] 스핀 락 평가	39
[9] Compare-And-Swap	40
[10] Load-Linked 그리고 Store-Conditional	41
[11] Fetch-And-Add	43
[12] 요약	45
[1-4] 병행성: 락 기반의 병행 자료 구조	46
[1] 병행 카운터	47
[2] 병행 연결리스트 - “락은 어디까지 걸어야 할까?”	54
[3] 병행 큐	61
[4] 병행 해시 테이블	65
[5] 요약	67
[1-5] 컨디션 변수	69
[1] 정의와 루틴들	71

[2] 생산자/소비자(유한 버퍼) 문제	75
-----------------------------	----

[1-1] 병행성: 개요

지금까지 운영체제가 다루는 기본 개념들의 발전 과정을 살펴보았다.

하나의 물리적 CPU를 다수의 가상 CPU로 확장하여 마치 여러 개의 프로그램이 동시에 실행하는 듯한 착시를 만들었다.

주소 공간(address space)이라는 개념을 통해 각 프로그램들이 마치 자신만의 메모리를 가지고 있는 것처럼 동작하는데, 사실은 운영체제가 물리 메모리를 여러 개의 주소 공간이 서로 번갈아 가면서 사용하게 한다.

이번 장에서는 프로세스를 위한 새로운 개념인 스레드(thread)를 소개한다.

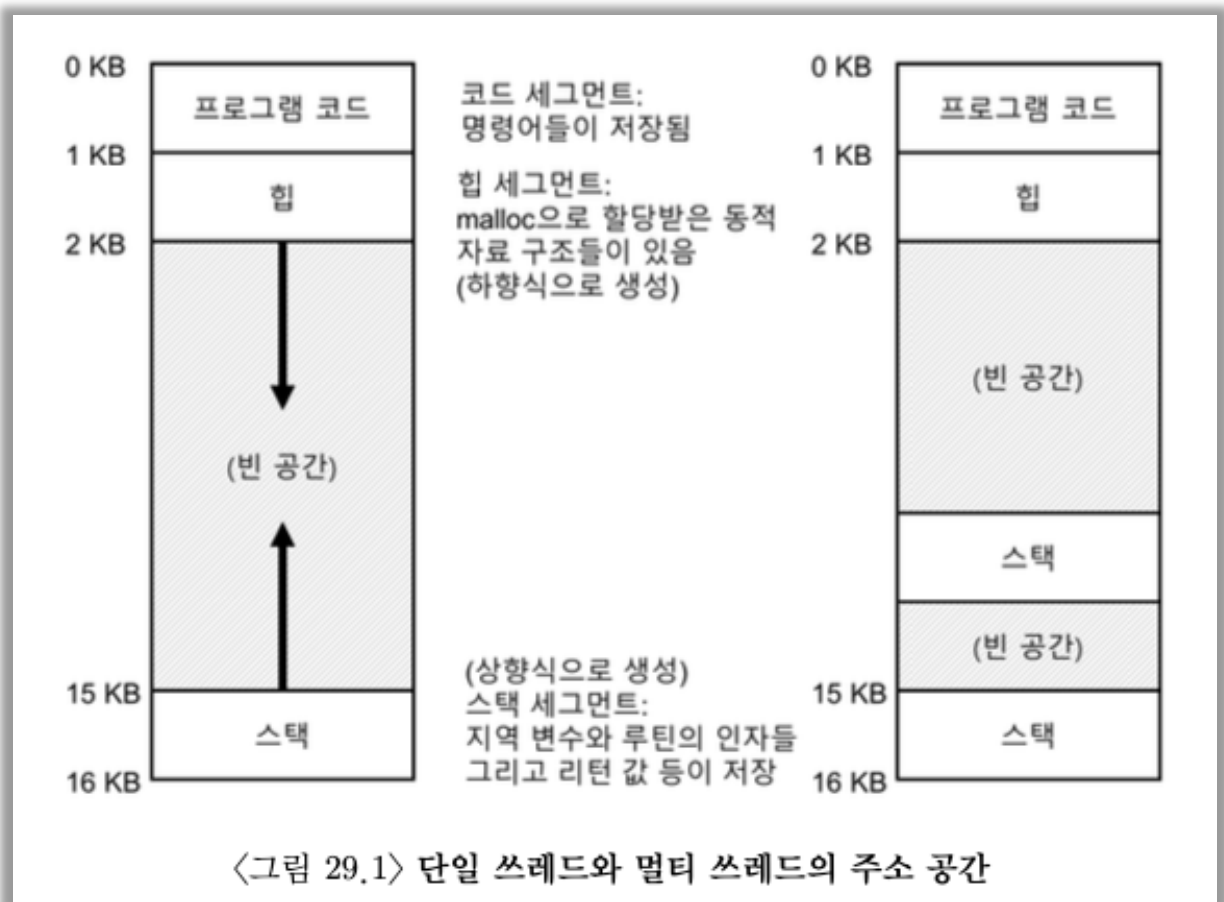
프로그램에서 한 순간에 하나의 명령어만 실행하는(단일 PC(Program Counter)값) 고전적인 관점에서 벗어나 멀티 스레드 프로그램은 하나 이상의 실행 지점(독립적으로 불러 들여지고 실행될 수 있는 여러 개의 PC 값)을 가지고 있다. 멀티 스레드를 이해하는 다른 방법은 각 스레드가 프로세스와 매우 유사하지만, 차이가 있으면 스레드들은 주소 공간을 공유하기 때문에 동일한 값에 접근할 수 있다는 것이다.

하나의 스레드의 상태는 프로세스의 상태와 매우 유사하다.

스레드는 어디서 명령어들을 불러 들일지 추적하는 프로그램 카운터(PC)와 연산을 위한 레지스터들을 가지고 있다.

만약 두 개의 스레드가 하나의 프로세서에서 실행 중이라면 실행하고자 하는 스레드(T2)는 반드시 문맥 교환(context switch)을 통해서 실행 중인 스레드(T1)와 교체되어야 한다.

스레드 간의 문맥 교환은 T1이 사용하던 레지스터들을 저장하고 T2가 사용하던 레지스터의 내요으로 복원한다는 점에서 프로세스의 문맥 교환과 매우 유사하다. 프로세스의 문맥 교환을 할 때에 프로세스의 상태를 프로세스 제어 블록(process control block, PCB)에 저장하듯이 프로세스의 스레드들의 상태를 저장하기 위해서는 하나 또는 그 이상의 스레드 제어 블록(thread control block, TCB)이 필요하다. 가장 큰 차이중 하나는 프로세스의 경우와 달리 스레드 간의 문맥 교환에서는 주소 공간을 그대로 사용한다는 것이다.(사용하고 있던 페이지 테이블을 그대로 사용하면 된다.)



쓰레드와 프로세스의 또 다른 차이는 스택에 있다.

고전적 프로세스 주소 공간과 같은 간단한 모델(단일 쓰레드 프로세스)에서는 스택이 하나만 존재한다.

(그림 좌측) 주로 주소 공간의 하부에 위치한다.

반면에 멀티 쓰레드 프로세스의 경우에는 각 쓰레드가 독립적으로 실행되며 쓰레드가 실행하기 위해 여러 루틴들을 호출할 수 있다.

주소 공간에는 하나의 스택이 아니라 쓰레드마다 스택이 할당되어 있다.

두 개의 쓰레드를 가지는 멀티 쓰레드 프로세스의 주소 공간은 단일 쓰레드 프로세스의 주소 공간과 다르다. (그림 우측)

이 그림에서는 두 개의 스택이 프로세스 주소 공간에 존재하는 것을 볼 수 있다.

스택에서 할당되는 변수들이나 매개변수, 리턴 값, 그리고 그 외에 스택에 넣는 것들은 해당 쓰레드의 스택인 쓰레드-로컬 저장소(thread-local storage)라 불리는 곳에 저장된다.

쓰레드-로컬 저장소로 인해서 정교한 주소 공간의 배치가 무너진다는 것을 알았을 것이다.

전에는 스택과 힙이 독립적으로 할당되기 때문에 주소 공간에 더 이상 공간이 없는 경우에만 문제가 생겼었다.

이제는 상황이 예전처럼 깔끔하지 않다.

다행스러운 것은 스택의 크기가 아주 크지 않아도 되기 때문에 대부분의 경우에는 무제가 되지 않는다.(재귀 호출을 아주 많이 하는 경우는 제외한다.)

[1] 예제: 쓰레드 생성

한 쓰레드는 “A”라고 출력하고 다른 쓰레드는 “B”라고 출력하는 독립적^o니 두 개의 쓰레드를 생성하는 프로그램을 실행시킨다고 해보자.

코드는 다음과 같다.

```
#include <stdio.h>
#include <assert.h>
#include <pthread.h>

void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    int rc;
    printf("main: begin\n");

    rc = pthread_create(&p1, NULL, mythread, "A");
    assert(rc == 0);
    rc = pthread_create(&p2, NULL, mythread, "B");
    assert(rc == 0);

    // 병렬 실행
    rc = pthread_join(p1, NULL);
    assert(rc == 0);
    rc = pthread_join(p2, NULL);
    assert(rc == 0);
    printf("main: end\n");
    return 0;
}
```

[1-1] 라이브러리 설명

#include <assert.h>

- 디버깅용 라이브러리
- 조건이 참이진 검사
- assert(조건)
 - 조건이 거짓이면 프로그램 강제 종료
 - 디버깅할 때 매우 유용

#include <pthread.h>

- POSIX 스레드 라이브러리
- 멀티스레딩을 위한 핵심 라이브러리
- 스레드 생성과 종료 대기, 동기화

[1-2] 함수 설명

void *mythread(void *arg)

```
void *mythread(void *arg){  
    printf("%s\n", (char*)arg);  
    return NULL;  
}
```

- 역할: 스레드가 실행할 함수
- 특징: pthread 규칙으로 반드시 형태가 아래와 같아야한다.
 - void *function(void *arg)
- 동작
 1. arg 를 문자열로 캐스팅
 2. 출력(A 또는 B)
 3. 종료(NULL 반환)

pthread_create(스레드 ID, 속성, 실행함수, 인자)

- rc = pthread_create(&p1, NULL, mythread, "A");
- &p1 : 생성된 스레드 ID 저장
- NULL : 기본 속성 사용
- mythread : 실행할 함수
- "A" : 전달할 인자

assert(rc == 0);

- 스레드 생성 성공 확인

pthread_join(스레드 ID, 반환값);

- pthread_join(p1, NULL);
- 역할: 해당 스레드가 끝날 때까지 기다림
- 중요 이유: main 이 먼저 끝나버리는 걸 방지

메인 프로그램은 각각 mythread() 함수를 실행할 두 개의 스레드를 생성한다.

이때 각 mythread() 함수는 서로 다른 인자를 전달받는다.

스케줄러의 동작에 따라 다르겠지만, 스레드가 생성되면, 즉시 실행될 수도 있고, 준비(Read)상태에서 실행은 되지 않을 수 있다.

두 개의 스레드(T1 과 T2)를 생성한 후에 메인 스레드는 pthread_join()을 호출하여 특정 스레드의 동작의 종료를 대기한다.

이 프로그램의 가능한 실행 순서를 살펴보도록 하자.

첫 번째 실행 흐름

1. main 시작
2. “main: begint” 출력
3. Thread1 생성
4. Thread2 생성
5. main → join(p1) → 기다림
 - 이때 OS 가 Thread1 실행
6. “A” 출력
7. Thread1 종료
 - 다시 main 진행
8. join(p2) → 기다림
 - Thread2 실행
9. ”B”출력
10. 종료
11. “main:end”

- 특징: 두 스레드가 뒤늦게 실행
 - main 이 먼저 쪽 진행
 - join 에서 멈춤

두 번째 실행 흐름

1. main 시작
2. “main: begint” 출력
3. Thread1 생성
 - 바로 Thread1 실행
4. “A” 출력
5. 종료
6. “Thread2 실행
- 바로 Thread2 실행
7. “B” 출력
8. 종료
9. main → join(p1)
 - 이미 끝났으니까 바로 리턴
10. join(p2)
 - 이미 끝남 → 바로 리턴
11. “main: end”

- 특징: 스레드가 먼저 끝나버림
 - main 보다 빨리 실행됨
 - join 은 그냥 확인만 하고 끝

유의할 점은 위 실행 순서가 스레드가 유일한 실행 가능 순서가 아니라는 것이다. 스케줄러가 특정 시점에 실행하는 스레드에 따라 다양한 순서가 있을 수 있다.

스레드의 생성이 함수의 호출과 유사하게 보인다.

함수 호출에서는 함수 실행 후에 호출자(caller)에게 리턴하는 반면에 스레드의 생성에서는 실행할 명령어들을 갖고 있는 새로운 스레드가 생성되고, 생성된 스레드는 호출자와는 별개로 실행된다.

스레드 생성 함수가 리턴되기 전에 스레드가 실행될 수 있고, 그보다 이후에 실행될 수도 있다.

여기에서 알 수 있듯이, 스레드는 일을 복잡하게 만든다. 이 간단한 예제에서도 어떤 스레드가 언제 실행되는지 알기 어렵다.

컴퓨터는 병행성이라는 주제가 아니더라도 충분히 어려운데, 불행하게도 이 문제 때문에 더 어려워진다.

[2] 훨씬 더 어려운 이유: 데이터 공유

앞의 간단한 쓰레드 예제를 통해 쓰레드의 생성 방법을 살펴보았고, 실행 순서는 스케줄러의 동작에 따라 바뀔 수 있다는 것을 보았다.

예제에서 나타나지 않은 부분은 쓰레드가 공유 데이터를 접근하기 위해 상호작용하는 과정이다.

위에 나와 있는 코드를 사용하여 전역 공유 변수를 갱신하는 두 개의 쓰레드에 대한 간단한 예제를 살펴보자.

코드를 보기 앞서 몇 가지 짚고 넘어가야 할 것이 있다.

첫 번째, 이 예제에서는 Stevens 가 [SR05] 제안하듯이 쓰레드를 생성하는 루틴과 조인하는 루틴이 실패할 경우 간단하게 종료하도록 래퍼 함수를 만들었다.

이 예제와 같이 간단한 프로그램은 에러 발생 여부만 관심이 있기에, 종료 외에 다른 처리는 하지 않는다.

그렇기 때문에 Pthread_create()는 단순히 pthread_create()를 호출하고 리턴 값이 0 인지 확인한다.

만약 0 이 아니면 Pthread_creat()는 에러 메시지를 출력하고 종료한다.

두 번째, 작업자 쓰레드를 위해 두 개의 독립된 함수를 구성하는 대신 하나의 단일 코드를 사용하였다.

메시지를 출력하기 전에 각 쓰레드가 다른 문자를 출력하도록 쓰레드에게 문자열 인자를 전달하였다.

마지막으로, 그리고 가장 중요한 것은 각 작업자가 무엇을 하려는지 알 수 있다는 것이다.

각 작업자 쓰레드는 반복문 안에서 공유 변수인 counter 에 수를 천만 번(1e7) 더한다.

결과적으로 최종적으로 얻으려는 값은 20,000,000 이다.

```
#include <stdio.h>
#include <pthread.h>

static volatile int counter = 0;

void *mythread(void *arg)
{
    printf("%s: begin\n", (char *) arg);

    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }

    printf("%s: done\n", (char *) arg);
    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t p1, p2;

    printf("main: begin (counter = %d)\n", counter);

    pthread_create(&p1, NULL, mythread, "A");
    pthread_create(&p2, NULL, mythread, "B");

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);

    printf("main: done with both (counter = %d)\n", counter);
    return 0;
}
```

전역 변수

```
static volatile int counter = 0;
```

- static
 - 이 파일 안에서만 사용 가능
 - 전역 변수이지만 외부에서 접근 불가
- volatile
 - 이 변수는 언제든지 바뀔 수 있으니 항상 메모리에서 직접 읽기
- 여러 스레드가 사용하는 공유 변수

스레드 함수

```
void *mythread(void *arg)
```

반복문

```
for(i = 0; i < 1e7; i++)
```

- $1e7 = 10^7 = 10,000,000$

프로그램 의도

이 코드의 목표

- 스레드 2 개 생성
- 각각 1 천만 번씩 counter 증가
- 총 증가 횟수 = 2 천만

즉, counter = 20000000

실제 결과

```
counter = 17382921  
counter = 19823412  
counter = 20000000 (가끔)
```

원인

- Race Condition(경쟁 조건)
- counter = counter + 1;

실제 동작 과정

1. counter 값을 읽음
2. +1 계산
3. 다시 저장

문제 상황(동시에 실행될 때)

Thread A

- counter = 5 읽음

Thread B

- counter = 5 읽음

- 둘 다 같은 값 읽음

- A 실행: $5 + 1 = 6$ 저장

- B 실행: $5 + 1 = 6$ 저장

결과

- 원래는 7 이 되어야 하는데 6 이 됨(하나 날라감)

[3] 제어 없는 스케줄링

왜 이런 현상이 발생하는지 이해하려면 counter 갱신을 위해서 컴파일러가 생성한 코드의 실행 순서를 이해해야 한다.

카운터는 단순히 (1)이라는 숫자를 더하려고 한다.

x86 에서 counter 를 증가하는 코드의 순서는 다음과 같다.

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

이 예제에서 사용하는 counter 변수의 위치의 주소는 0x8049a1c 라고 가정한다.

이 세줄의 명령문에서는 먼저 x86 의 mov 명령어가 명시한 메모리 주소의 값을 읽어들인 후 eax 레지스터에 넣는다.

그리고 1(0x1)을 eax 레지스터의 값에 더하는 연산을 한 후에 마지막으로 eax 에 저장되어 있는 값을 메모리의 원래 주소에 다시 저장한다.

두 개의 쓰레드 중에 쓰레드 1 이 counter 를 증가시키는 코드 영역에 진입하여 counter 의 값을 1 증가시키려는 상황을 가정해보자.

counter 에 있는 값이 50 이었다고 하면 50 을 eax 레지스터에 넣는다.

쓰레드 1 에 있어서 $eax=50$ 이 된다. 그 후에 레지스터의 값에 1 을 더하여 $eax=51$ 이 된다.

이제 상황이 안좋아진다.

타이머 인터럽트가 발생하여 운영체제가 실행 중인 쓰레드의 PC 값과 eax 를 포함하는 레지스터들 등의 현재 상태를 쓰레드의 TCB 에 저장한다.

그리고는 상황이 더 악화된다.

쓰레드 2가 선택되고 counter 값을 증가시키는 똑같은 코드 영역에 진입한다.

첫 번째의 명령어를 실행하여 counter 값을 얻어 온 후 eax에 넣는다.

쓰레드는 개별적으로 쓰레드 전용 레지스터를 가지고 있다.

사용 중이던 레지스터들을 저장하고 복구하는 문맥 교환 코드에 의해 이 레지스터들은 가상화된다.

counter 값은 아직 50을 나타내고 있어서 쓰레드 2의 eax의 값은 50이다.

쓰레드 2가 그 다음의 두 문장을 실행한다면 eax 값을 1 증가시키고(eax=51) 난 후에 eax 값을 counter(주소 0x8049a1c)에 저장한다.

전역 변수인 counter는 이제 51이라는 값을 가진다.

최종적으로 또 한 번의 문맥 교환이 발생하면 쓰레드 1이 리턴하여 실행된다.

이전 상황을 기억해 보면 쓰레드 1은 mov와 add 동작을 실행하였고 이제 마지막의 mov 명령어를 수행하려는 중이다.

그리고 eax=51이었다.

그렇기 때문에 mov 명령어가 실행되면 메모리에 레지스터의 값을 저장하여 counter의 값을 증가시키는 코드가 두 번 수행이 되었지만 50에서 시작한 counter의 값은 1 증가한 51이다.

“정확하게” 동작하도록 작성된 프로그램이라고 한다면 counter의 값은 52가 되어야 한다.

이 문제를 좀 더 잘 이해하기 위해서 실행의 흐름을 구체적으로 살펴보자.

이 예제에서는 다음에 나타난 것과 같이 코드가 메모리 주소에 100에 저장되었다고 가정해 보자.

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

counter의 값은 50에서 시작한다고 가정하고 예제의 내용을 상기하며 따라가 보자. 예시에서처럼 명령어의 실행 순서에 따라 결과가 달라지는 상황을 경쟁 조건(race condition)이라고 부른다.

문맥 교환이 때에 맞지 않게 실행되는 운이 없는 경우 잘못된 결과를 얻게 된다.

사실, 경쟁 조건에 처한 경우 실행할 때마다 다른 결과를 얻는다.

컴퓨터의 작동에서 일반적으로 발생하는 결정적 결과와 달리 결과가 어떠할지 알지 못하거나 실행할 때마다 결과가 다른 경우를 비결정적(indeterminate)인 결과라고 부른다.

멀티 쓰레드 같은 코드를 실행할 때 경쟁 조건이 발생하기 때문에 이러한 코드 부분을 임계 영역(critical section)이라고 부른다.

공유 변수(또는 더 일반적으로는 공유 자원)를 접근하고 하나 이상의 쓰레드에서 동시에 실행되면 안 되는 코드를 임계 영역이라고 부른다.

이러한 코드에서 필요한 것은 상호 배제(mutual exclusion)이다.

이 속성은 하나의 쓰레드가 임계 영역 내의 코드를 실행 중일 때는 다른 쓰레드가 실행할 수 없도록 보장해준다.

OS	Thread 1	Thread 2	(명령어 실행 후)		
			PC	%eax	counter
	임계 영역 전		100	0	50
	mov 0x8049a1c, %eax		105	50	50
	add \$0x1, %eax		108	51	50
	interrupt				
	T1 상태 저장				
	T2 상태 복원		100	0	50
		mov 0x8049a1c, %eax	105	50	50
		add \$0x1, %eax	108	51	50
		mov %eax, 0x8049a1	113	51	51
	interrupt				
	T2 상태 저장				
	T1 상태 복원		108	51	51
	mov %eax, 0x8049a1c		113	51	51

[3] 원자성에 대한 바람

임계 영역 문제에 대한 해결 방법 중 하나로 강력한 명령어 한 개로 의도한 동작을 수행하여, 인터럽트 발생 가능성을 원천적으로 차단하는 것이다.

예를 들면, 다음과 같은 강력한 명령어가 있다고 가정해 보자.

memory-add 0x8049a1c, \$0x1

이 명령어는 메모리 상의 위치에 어떤 값을 더하는 명령어다.

하드웨어는 이 명령어가 원자적으로 실행되는 것을 보장한다고 하자.

이 명령어가 실행되면 원하는 것처럼 값이 갱신될 것이다.

하드웨어가 원자성을 보장해주기 때문에 명령어 수행 도중에 인터럽트가 발생하지 않는다.

인터럽트가 발생하더라도, 명령어가 실행이 안되었거나 실행이 종료된 후라는 것을 의미하고 그 중간 상태는 있을 수 없다.

멋진 하드웨어이다.

문맥 상으로 원자적이라는 말은 “하나의 단위”를 뜻하며 때로는 “전부 아니면 전무”로 이해될 수도 있다. 다음의 세 개의 명령어가 원자적으로 실행되기를 원한다.

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

언급했듯이, 위의 명령어들을 하나의 명령어로 대신할 수 있다면 그 명령어를 사용하면 그만이다.

하지만 일반적인 상황에서는 그러한 명령어가 없다고 봐야 한다.

병행성을 가지는 B-tree를 만드는 중이고, 값을 갱신한다고 했을 때, 원자적으로 B-tree를 갱신하는 어셈블리 명령어를 원할까?

어떤 작업을 수행할 때, 만약 여러 개의 명령어를 하나로 대신할 수 있는 단일 명령어가 있다면 그것을 사용하는 것이 가장 간단하고 안전한 방법이다.

하지만 현실에서는 대부분의 경우 그런 “완전히 원자적인 하나의 명령어”는 존재하지 않는다. 예를 들어, 병행성을 고려한 B-tree를 구현하면서 값을 갱신한다고 했을 때, 전체 B-tree 갱신 작업을 한 번에 처리해주는 어셈블리 명령어가 있을 것이라고 기대하는 것은 비현실적이다.

대신 하드웨어는 아주 기본적인 수준의 동기화 명령어들만 제공한다.

이러한 명령어들은 직접적으로 복잡한 자료구조를 처리하지는 않지만, 이를 바탕으로 락(lock)이나 세마포어 같은 동기화 도구를 구현할 수 있다.

그리고 운영체제는 이러한 하드웨어 기능을 활용하여 여러 스레드가 동시에 실행되는 환경에서도 올바르게 동작하도록 지원한다.

결국 우리는 하드웨어가 제공하는 최소한의 동기화 기능과 운영체제의 도움을 이용해, 여러 스레드 중 오직 하나만이 특정 코드 영역(임계 영역)에 들어가도록 제어할 수 있다. 이를 통해 병행 실행 환경에서도 안정적으로 동작하는 멀티스레드 프로그램을 작성할 수 있다.

이 과정은 결코 쉽지 않다.

병행성 문제는 본질적으로 복잡하고 이해하기 어렵기 때문에, 공부를 하다보면 머리가 아플 정도로 헷갈릴 수 있다.

하지만 그런 상태에 도달했다는 것은 오히려 개념을 제대로 이해해 가고 있다는 신호일 수 있다.

너무 어렵게 느껴질 때는 잠시 쉬어가면서 천천히 익히는 것이 중요하다.

여담: 병행성 관련 핵심 용어 정리

1. 임계 영역(Critical Section)

- 여러 스레드가 공유 자원(변수, 자료 구조 등)에 접근하는 코드 영역
- 동시에 여러 스레드가 들어가면 문제가 발생할 수 있음

2. 경쟁 조건(Race Condition)

- 여러 스레드가 거의 동시에 임계 영역에 진입하려 할 때 발생
- 공유 데이터를 동시에 수정하면서 예상하지 못한 결과가 나옴

3. 비결정적(Indeterminate)

- 실행할 때마다 결과가 달라지는 프로그램 상태
- 원인: 스레드 실행 순서가 매번 달라짐(경쟁 조건 포함)

4. 상호 배제(Mutual Exclusion)

- 한 번에 하나의 스레드만 임계 영역에 들어가도록 제한하는 기법
- 경쟁 조건을 방지하고 일관된 결과 보장

[4] 또 다른 문제: 상대 기다리기

이제까지 병행성 문제를 공유 변수 접근에 관련된 스레드 간의 상호 작용 문제로 정의하였다.

실제로 하나의 스레드가, 다른 스레드가 어떤 동작을 끝낼 때까지 대기해야 하는 상황도 빈번하게 발생한다.

프로세스가 디스크 I/O 를 요청하고 응답이 올 때까지 잠든 경우가 좋은 예이다. I/O 완료 후 잠들었던 프로세스가 깨어나 이후 작업을 진행한다.

이후에서는 원자성 지원을 위한 동기화 함수 제작에 대한 내용과 멀티 스레드 프로그램에서 흔한 잠자기/깨우기 동작에 대한 지원 기법에 대해서 다룬다.

지금 이해가 가지 않더라도 괜찮다.

컨디션 변수(condition variable)에 대한 장을 읽을 때 즈음에는 이해가 될 것이다.

[6] 정리: 왜 운영체제에서?

운영체제에서 병행성 문제를 다루는 이유는 한마디로 “역사” 때문이다.

운영체제는 가장 먼저 등장한 병행 프로그램이었고, 이 안에서 다양한 동기화 기법들이 개발되었다.

이후 멀티스레드 프로그램이 등장하면서 일반 개발자들도 같은 문제를 다루게 되었다.

예를 들어, 두 프로세스가 동시에 파일에 `write()`를 수행하면 파일의 길이와 inode 같은 공유 데이터를 함께 수정하게 된다.

이 과정에서 인터럽트가 발생하면 데이터가 꼬일 수 있기 때문에, 이러한 코드는 임계 영역이 된다.

결국 운영체제 내부의 자료구조(페이지 테이블, 프로세스 리스트, 파일 시스템 등)는 항상 여러 실행 흐름에 의해 영향을 받을 수 있기 때문에, 이를 안전하게 관리하려면 동기화 기법이 필수적이다.

[1-2] 병행성: 쓰레드 API

핵심 질문: 쓰레드를 생성하고 제어하는 방법

- 운영체제가 쓰레드를 생성하고 제어하는 데 어떤 인터페이스를 제공해야 할까?
- 어떻게 이 인터페이스를 설계해야 쉽고 유용하게 사용할 수 있을까?

이번 장에서는 쓰레드 API의 주요 부분을 간략하게 다룬다.

각 부분에 대한 자세한 내용은 이후의 장에서 다루도록 한다.

이후의 장들에서 컨디션 변수와 락에 대해 예제와 함께 천천히 소개할 것이다.

이번 장은 참고용으로 사용하면 좋겠다.

[1] 쓰레드 생성

멀티 쓰레드 프로그램을 작성할 때 가장 먼저 해야 할 일은 새로운 쓰레드를 생성하는 것이다.

이를 위해 POSIX에서는 `pthread_create()` 함수를 제공하며, 이 함수를 통해 쉽게 쓰레드를 만들 수 있다.

```
#include <pthread.h>

int pthread_create(
    pthread_t *thread,
    const pthread_attr_t *attr,
    void *(*start_routine)(void *),
    void *arg
);
```

이 함수는 총 네 개의 인자를 가진다.

- `thread`:

`pthread_t` 타입의 구조체를 가리키는 포인터로, 생성된 쓰레드를 식별하고 이후에 제어할 때 사용된다. 쓰레드를 생성할 때 반드시 이 변수를 전달해야 한다.

- `attr`:

쓰레드의 속성을 설정하는 데 사용되며, 스택 크기나 스케줄링 우선순위 등을 지정할 수 있다. 하지만 대부분의 경우 기본값이면 충분하기 때문에 `NULL`을 전달한다.

- `start_routine`

쓰레드가 실제로 실행할 함수이다. 이 함수는 다음과 같은 형태를 가져야 한다.

```
void *function(void *arg)
```

즉, `void *`타입의 인자를 받아서 `void *`타입을 반환해야 한다.

처음에는 복잡해 보일 수 있지만, 이렇게 하면 어떤 타입의 데이터든 유연하게 전달하고 반환할 수 있다는 장점이 있다.

- arg

쓰레드 함수에 전달할 인자이다. 여러 개의 값을 전달해야 할 경우에는 구조체로 묶어서 전달하는 것이 일반적이다.

다음 구조체를 이용해 여러 값을 전달하는 예제이다.

```
#include <pthread.h>
#include <stdio.h>

typedef struct {
    int a;
    int b;
} myarg_t;

void *mythread(void *arg) {
    myarg_t *m = (myarg_t *) arg;
    printf("%d %d\n", m->a, m->b);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    int rc;

    myarg_t args;
    args.a = 10;
    args.b = 20;

    rc = pthread_create(&p, NULL, mythread, &args);

    return 0;
}
```

이 코드에서는 myarg_t 구조체를 사용하여 두 개의 정수 값을 하나로 묶어 전달한다. 쓰레드 함수에서는 전달받은 void *타입의 인자를 다시 원래 타입으로 캐스팅하여 값을 꺼내 사용한다.

이렇게 쓰레드를 생성하면, 해당 쓰레드는 자신만의 스택을 가지면서도 기존 프로그램과 동일한 주소 공간을 공유하는 또 하나의 실행 흐름이 된다. 즉, 하나의 프로그램 안에서 여러 개의 쓰레드가 동시에 실행되는 구조가 만들어진다.

이제 프로그램은 단순한 순차 실행을 넘어, 여러 작업을 동시에 처리하는 병행 실행 환경으로 확장된다.

이후에는 이러한 쓰레드들이 함께 실행되면서 발생하는 다양한 문제(동기화, 경쟁 조건 등)를 다루게 되며, 이것이 멀티 쓰레드 프로그래밍의 핵심이다.

[2] 쓰레드 종료

쓰레드를 생성한 이후에는, 다른 쓰레드의 작업이 끝날 때까지 기다려야 하는 상황이 생길 수 있다.

이때 사용하는 함수가 `pthread_join()`이다.

이 함수는 특정 쓰레드가 종료될 때까지 기다리고, 필요하다면 그 쓰레드가 반환값도 받아올 수 있다.

```
#include <pthread.h>
```

```
int pthread_join(pthread_t thread, void **value_ptr);
```

이 함수는 두 개의 인자를 가진다.

첫 번째 인자인 `thread` 는 기다릴 쓰레드를 지정하는 값으로, `pthread_creat()`를 통해 생성할 때 초기화된 `pthread_t` 변수를 사용한다.

이 값을 보관해 두었다가 나중에 해당 쓰레드가 끝날 때까지 기다릴 때 사용한다.

두 번째 인자인 `value_ptr` 는 쓰레드의 반환값을 저장하기 위한 포인터이다.

쓰레드는 `void *` 타입으로 값을 반환할 수 있기 때문에, 어떤 타입이든 받을 수 있도록 이중 포인터(`void **`)형태로 전달한다.

중요한 점은, 값 자체가 아니라 값을 저장할 변수의 주소를 전달해야 한다는 것이다.

다음에 쓰레드에서 값을 반환하고, 메인 쓰레드에서 이를 받아오는 예제이다.

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
typedef struct {
    int a;
    int b;
} myarg_t;

typedef struct {
    int x;
    int y;
} myret_t;

void *mythread(void *arg) {
    myarg_t *m = (myarg_t *) arg;
    printf("%d %d\n", m->a, m->b);
    myret_t *r = malloc(sizeof(myret_t));
    r->x = 1;
    r->y = 2;

    return (void *) r;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    myret_t *m;

    myarg_t args;
    args.a = 10;
    args.b = 20;

    pthread_create(&p, NULL, mythread, &args);
    pthread_join(p, (void **) &m);
    printf("returned %d %d\n", m->x, m->y);
    return 0;
}
```

이 코드에서는 구조체를 이용해 인자를 전달하고, 또 다른 구조체를 이용해 결과를 반환한다.

메인 스레드는 pthread_join()에서 기다리다가, 스레드가 종료되면 반환값을 받아 사용할 수 있다.

여기서 몇 가지 중요한 점이 있다.

첫째, 항상 구조체로 묶어서 전달해야 하는 것은 아니다.

인자가 필요 없으면 NULL을 전달할 수 있고, 반환값이 필요 없으면 pthread_join()의 두 번째 인자도 NULL로 설정할 수 있다.

둘째, 하나의 값만 전달할 경우에는 구조체 없이 간단하게 처리할 수도 있다.

```
#include <stdio.h>
#include <pthread.h>

void *mythread(void *arg) {
    int m = (int) arg;
    printf("%d\n", m);
    return (void *) (arg + 1);
}

int main() {
    pthread_t p;
    int m;

    pthread_create(&p, NULL, mythread, (void *) 100);
    pthread_join(p, (void **) &m);

    printf("returned %d\n", m);
    return 0;
}
```

셋째, 가장 중요한 점은 쓰레드에서 반환하는 값의 메모리 위치이다.
아래와 같은 코드는 매우 위험하다.

```
void *mythread(void *arg) {  
    myret_t r;  
    r.x = 1;  
    r.y = 2;  
    return (void *) &r; // ❌ 위험  
}
```

이 코드는 함수 내부의 지역 변수(스택 영역)에 대한 주소를 반환한다.
하지만 쓰레드가 종료되면 해당 스택 메모리는 해제되므로, 반환된 포인터는 더 이상 유효하지 않다.
이런 코드는 잘못된 값이나 예기치 않은 동작을 유발할 수 있다.

마지막으로, 쓰레드를 생성하자마자 바로 pthread_join()을 호출하는 것은 좋은 사용 방법이 아니다.

이렇게 하면 실제로는 순차 실행과 다를 바 없기 때문이다.
일반적으로는 여러 개의 쓰레드를 먼저 실행한 후, 나중에 한 번에 기다리는 방식으로 병렬성을 활용한다.

또한 모든 프로그램이 pthread_join()을 사용하는 것은 아니다.
예를 들어, 웹 서버처럼 계속 실행되는 프로그램은 쓰레드를 계속 생성하고 작업을 처리하기 때문에 종료를 기다릴 필요가 없다.
반면, 병렬 계산을 수행하는 프로그램에서는 모든 쓰레드가 작업을 마칠 때까지 기다린 후 다음 단계로 진행하기 위해 pthread_join()을 사용한다.

[3] 락(Lock)

쓰레드 생성과 종료 다음으로 중요한 개념은 락(lock)이다.

락은 여러 쓰레드가 동시에 공유 자원에 접근하는 것을 막기 위해, 즉 임계 영역을 보호하기 위해 사용된다.

POSIX 에디션 다음 두 가지 기본 함수로 락을 사용한다.

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

기본적인 사용 형태는 다음과 같다.

```
pthread_mutex_t lock;  
  
pthread_mutex_lock(&lock);  
x = x + 1;    // 임계 영역  
pthread_mutex_unlock(&lock);
```

이 코드는 다음과 같이 동작한다.

- 어떤 스레드가 lock()을 호출했을 때, 아무도 락을 가지고 있지 않으면 바로 획득하고 임계 영역에 진입한다.
- 이미 다른 스레드가 락을 가지고 있다면, 락을 얻을 때까지 대기한다.
- 락을 가진 스레드만 unlock()을 호출할 수 있다.

락 사용시 주의점

1. 초기화 필요

락은 반드시 초기화해야 한다.

정적 초기화

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

동적 초기화

```
pthread_mutex_t lock;  
int rc = pthread_mutex_init(&lock, NULL);
```

사용이 끝나면 pthread_mutex_destroy()도 호출해야 한다.

2. 에러 체크 필요

락 관련 함수도 실패할 수 있기 때문에 반드시 확인해야 한다.

```
void Pthread_mutex_lock(pthread_mutex_t *mutex) {  
    int rc = pthread_mutex_lock(mutex);  
    assert(rc == 0);  
}
```

3. 추가 락 함수

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);  
int pthread_mutex_timedlock(pthread_mutex_t *mutex, struct timespec *timeout);
```

- trylock: 락이 이미 사용 중이면 바로 실패
 - timedlock: 일정 시간 기다리다가 실패
- 보통은 잘 사용하지 않지만, 무한 대기 방지에 활용됨

[4] 컨디션 변수(Condition Variable)

컨디션 변수는 스레드 간에 “조건이 만족될 때까지 기다렸다가 깨우는”메커니즘이다.

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);  
int pthread_cond_signal(pthread_cond_t *cond);
```

- 컨디션 변수는 반드시 락과 함께 사용해야 한다.

기본 사용 패턴

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;  
  
pthread_mutex_lock(&lock);  
while (ready == 0)  
    pthread_cond_wait(&cond, &lock);  
pthread_mutex_unlock(&lock);
```

- 조건(ready == 0)이 만족되지 않으면 잠듦
- 다른 스레드가 깨워줄 때까지 대기

깨우는 코드

```
pthread_mutex_lock(&lock);  
ready = 1;  
pthread_cond_signal(&cond);  
pthread_mutex_unlock(&lock);
```

중요한 특징

1. wait 는 락을 자동으로 반납

- pthread_cond_wait()는
 - 잠들면서 락을 풀고
 - 깨어나면서 다시 락을 획득

그래야 다른 스레드가 들어와서 조건을 바꾸고 signal 가능

2. 반드시 while 사용

```
while (ready == 0)  
    pthread_cond_wait(&cond, &lock);
```

- 깨어났다고 해서 조건이 반드시 만족된 것은 아님
- 다시 검사해야 안전

3. signal 은 “힌트”일 뿐

- signal 은 “조건이 바뀌었을 수도 있으니 확인해봐” 정도의 의미

잘못된 방법

```
while (ready == 0)  
    ;    // busy waiting
```

- CPU 낭비(계속 반복)
- 버그 발생 가능성 높음

[5] 컴파일과 실행

이 장에서 다른 예제들은 비교적 간단하게 실행할 수 있다.

멀티 스레드 프로그램을 컴파일하려면 반드시 pthread 라이브러리를 함께 링크해야 한다.

이를 위해 두 가지가 필요하다.

1. pthread.h 헤더 포함
2. 컴파일 시 -pthread 옵션 추가

예를 들어, 다음과 같이 컴파일하면 된다.

```
gcc -o main main.c -Wall -pthread
```

이 명령어는

- main.c 파일을 컴파일하고
- pthread 라이브러리를 연결하여
- 실행 파일 main 을 생성한다.

중요한 점

- 코드에 #include <pthread.h>가 포함되어 있어야 정상적으로 컴파일된다.

단, 컴파일이 성공했다고 해서 프로그램이 항상 올바르게 동작하는 것은 아니다.

멀티 스레드 프로그램은 실행 순서에 따라 결과가 달라질 수 있기 때문에, 정확한 동작을 보장하는 것은 또 다른 문제이다.

[6] 요약

이 장에서 다음과 같은 pthread 의 핵심 개념들을 살펴보았다.

- 스레드 생성(pthread_create)
- 스레드 종료 대기(pthread_join)
- 락을 이용한 상호 배제(pthread_mutex)
- 컨디션 변수를 이용한 동기화(pthread_cond)

이러한 기능들을 활용하면 멀티 스레드 프로그램을 작성할 수 있지만, 올바르게 효율적인 코드를 작성하는 것은 쉽지 않다.

멀티 스레드 프로그래밍에서는

- 작은 실수도 큰 문제로 이어질 수 있기 때문에
- 인내와 세심한 주의가 매우 중요하다.

또한, pthread API 에는 이 외에도 많은 기능들이 존재한다.
리눅스에서는 다음 명령어로 다양한 API 를 확인할 수 있다.

```
man -k pthread
```

멀티 쓰레드에서 어려운 점은 API 자체가 아니라, 병행 상황을 정확하게 이해하고 설계하하는 사고 방식이다.

[1-3] 병행성: 락

병행성에 대한 소개 이후 병행 프로그램의 근본적인 문제 몇 개를 살펴보았다. 여러 개의 명령어들을 원자적으로 실행해보고 싶지만 단일 프로세서의 인터러블로 인해서(또는 멀티 쓰레드를 여러 프로세서에 병행성하려고 해서) 그렇게 할 수가 없다.

이 장에서는 앞서 다룬 락(lock)을 이용하여 이 문제를 직접적으로 다루고자 한다. 프로그래머들은 소스 코드의 임계 영역을 락으로 둘러서 그 임계 영역이 마치 하나의 원자 단위 명령어인 것처럼 실행되도록 한다.

[1] 락: 기본 개념

예를 위해 다음의 임계 영역이 있다고 하자. 고전적인 공유 변수의 갱신이다.

```
balance = balance + 1;
```

연결 리스트에 노드를 삽입하거나 공유 구조에 복잡한 갱신과 같은 다른 임계 영역을 사용할 수 있겠지만 간단한 예제를 사용하도록 하겠다.

락을 사용하기 위해서 락으로 임계 영역을 다음과 같이 감쌌다.

```
lock_t mutex; // 글로벌 변수로 선언된 락
...
lock(&mutex);
balance = balance + 1;
unlock(&mutex);
```

락은 하나의 변수이기 때문에 락을 사용하기 위해서는 락 변수를(위에 사용한 mutex 와 같이) 먼저 선언해야 한다.

이 락 변수는 (또는 짧게 락은) 락의 상태를 나타낸다.

이 락은 사용 가능(available) 상태이거나(해제(unlocked 또는 free), 즉 어느 쓰레드도 락을 갖고 있지 않거나, 사용 중(acquired), 즉 임계 영역에서 정확히 하나의 쓰레드가 락을 획득한 상태이다.

이 락 자료 구조에 락을 보유한 쓰레드에 대한 정보나 락을 대기하는 쓰레들에 대한 정보를 저장할 수 있다.

물론, 이러한 정보는 락 사용자는 알 수 없다.

lock()과 unlock() 루틴의 의미는 간단하다.

lock() 루틴 호출을 통해 락 획득을 시도한다.

만약 어떤 쓰레드도 락을 갖고 있지 않으면 그 쓰레드는 락을 획득하여 임계 영역 내로 진입한다.

이렇게 진입한 쓰레드를 락 소유자(owner)라고 부른다.

만약 다른 쓰레드가 lock()을 호출한다면(이 예제에서는 mutex 에 해당) 사용 중인 동안에는 lock() 함수가 리턴하지 않는다.

이와 같은 방식으로 락을 보유한 쓰레드가 임계 영역에 진입한 상태에는 다른 쓰레드들이 임계 영역 안으로 진입할 수가 없다.

락 소유자가 unlock()을 호출한다면 락은 이제 다시 사용 가능한 상태가 된다.

어떤 쓰레드도 이 락을 대기하고 있지 않았다면(어떤 쓰레드도 lock()을 호출하여 멈춰 있던 상태가 아니라면) 락의 상태는 사용 가능으로 유지된다.

만약에 대기 중이던 쓰레드가 있었다면(lock()으로 인해 멈춘), 락의 상태가 변경되었다는 것을 (결국엔) 인지하고 (또는 정보를 받아서) 락을 획득하여 임계 영역 내로 진입하게 된다.

락은 프로그래머에게 스케줄링에 대한 최소한의 제어권을 제공한다.

일반적으로 쓰레드는 프로그래머가 생성하고 운영체제가 제어한다.

락은 쓰레드에 대한 제어권을 일부 이양 받을 수 있게 해준다.

락으로 코드를 감싸서 프로그래머는 그 코드 내에서는 하나의 쓰레드만 동작하도록 보장할 수 있다.

락을 통해 프로세스들의 혼란스런 실행 순서에 어느 정도의 질서를 부여할 수 있다.

[2] Pthread 락

쓰레드 간에 상호 배제(mutual exclusion) 기능을 제공하기 때문에 POSIX

라이브러리는 락을 mutex 라고 부른다.

상호 배제는 한 쓰레드가 임계 영역 내에 있다면 이 쓰레드의 동작이 끝날 때까지 다른 쓰레드가 임계 영역에 들어 올 수 없도록 제한한다고 해서 얻은 이름이다.

다음과 같은 POSIX 쓰레드 코드를 만나면 앞에서 언급한 것과 같은 동작을 한다고 이해하면 된다.

```
pthread_mutex_t lock = PTHREAD_INITIALIZER;

pthread_mutex_lock(&lock); // pthread_mutex_lock()을 위한 래퍼
balance = balance + 1;
pthread_mutex_unlock(&lock);
```

POSIX 방식에서는 변수 명을 지정하여 락과 언락 함수에 전달하고 있다.
다른 변수를 보호하기 위해서 다른 락을 사용할 수도 있기 때문이다.
이러한 방식을 통해 동시에 더 많은 일을 처리할 수 있다.
하나의 락이 임의의 임계 영역에 진입할 때마다 사용하는 것이 아니라(거친(coarse-grained) 락 사용 전략), 서로 다른 데이터와 자료 구조를 보호하기 위해 여러 락을 사용하여 한 번에 여러 스레드가 서로 다른 락으로 보호된 코드 내에 각자가 진입이 가능하도록 할 수 있다.(세밀한(fine-grained) 락 사용 전략)

[3] 락 구현

락의 동작을 프로그래머 입장에서 대략 이해했을 것이다.

핵심 질문: 락은 어떻게 만들까

- 효율적인 락은 어떻게 만들어야 하는가?
- 효율적인 락은 낮은 비용으로 상호 배제 기법을 제공하고 다음에 다룰 몇 가지 속성들을 추가로 가져야 한다.
- 어떤 하드웨어 자원이 필요한가?
- 어떤 운영체제 지원이 필요한가?

사용 가능한 락을 만들기 위해서는 우리의 오래된 친구들인 하드웨어와 운영체제의 도움을 받아야 한다.

오랜 기간을 걸쳐 다양한 컴퓨터 구조의 명령어 집합에 여러 하드웨어 명령어들이 추가되었다.

이러한 명령어들이 어떻게 구현되었는지는 컴퓨터 구조 수업의 주제이기 때문에 다루지 않겠지만, 락과 같은 상호 배제를 위한 기법을 만드는 데 어떻게 활용하는지는 학습할 것이다.

정교한 락 라이브러리를 제작하는 데 있어 운영체제가 관여하는 것은 무엇인지 알아보겠다.

[4] 락의 평가

어떤 락이든 만들기 전에 첫째로 목표를 이해해야 하고 구현의 효율을 어떻게 평가할지 질문해야 한다.

락이 동작하는지(잘 동작하는지) 평가하기 위해 먼저 기준을 정해야 한다.

첫째는 상호 배제를 제대로 지원하는 가이다.

기본적으로 락이 동작하여 임계 영역 내로 다수의 스레드가 진입을 막을 수 있는지 검사해야 한다.

둘째는 공정성(fairness)이다. 스레드들이 락 획득에 대한 공정한 기회가 주어지는가? 이것을 보는 다른 관점은 좀 더 극단적인 상황을 평가해 보는것이다.

락을 전혀 얻지 못해 굶주리는(startv) 경우가 발생하는가?

마지막 기준은 성능(preformance)이다.

특히, 락 사용 시간전 오버헤드를 평가해야 한다.

이 주제에 대해서 고려해야 할 몇 가지 사항이 있다.

- 하나는 경쟁이 전혀 없는 경우의 성능이다.

하나의 스레드가 실행 중에 락을 획득하고 해제하는 과정에서 발생하는 부하는 얼마나 되는가?

- 다음은 여러 스레드가 단일 CPU 상에서 락을 획득하려고 경쟁할 때의 성능이다.

- 마지막은 멀티 CPU 상황에서 락 경쟁 시의 성능이다.

이런 서로 다른 상황의 성능을 평가해야 하므로 앞으로 다룰 다양한 락 기법들이 성능에 미치는 영향을 이해할 수가 있다.

[5] 인터럽트 제어

초창기 단일 프로세서 시스템에서는 상호 배제 지원을 위해 임계 영역 내에서는 인터럽트를 비활성화하는 방법을 사용했었다. 코드는 다음과 같다.

```
void lock() {  
    DisableInterrupts();  
}  
  
void unlock() {  
    EnableInterrupts();  
}
```

그렇게 동작하는 단일 프로세서를 사용한다고 가정해 보자.

임계 영역에 진입하기 전에 특별한 하드웨어 명령어를 사용하여 인터럽트를 막는다면 임계 영역 내의 코드에서는 인터럽트가 발생할 수 없기 때문에 원자적으로 실행될 수 있다.

모든 동작이 끝난 후에 다시 하드웨어 명령어를 사용하여 인터럽트를 사용 가능하도록 하여 프로그램이 이전처럼 진행할 수 있도록 한다.

이 방법의 주요 장점은 단순하다는 것이다.

이 방법이 왜 제대로 동작하는지 깊이 생각하지 않아도 된다.

인터럽트가 발생하지 않으면, 코드가 실행 중에 다른 스레드가 중간에 끼어들지 않는다는 것을 보장할 수 있다.

이 방법은 단점이 많다.

먼저 이 요청을 하는 스레드가 인터럽트를 활성화/비활성화하는 특권(privileged) 연산을 실행할 수 있도록 허가해야 한다.

또, 이를 다른 목적으로 사용하지 않음을 신뢰할 수 있어야 한다.

운영체제가 잘 알지 못하는 다른 프로그램을 신뢰해야 하는 경우가 생긴다면, 대부분 곤경에 빠진 것이라고 보면 된다.

몇 가지 경우의 형태를 살펴보자.

첫 번째, 탐욕적(Greedy) 기법을 사용한 프로그램이 시작과 동시에 lock()을 호출하여 프로세서를 독점하여 사용할 수도 있다.

더한 경우라면 오류가 있거나 악의적인 프로그램이 lock()을 호출하고 무한 반복문에 들어갈 수도 있다.

후자의 경우라면 운영체제는 시스템의 제어권을 다시 얻을 수가 없다.

동기화를 위해 인터럽트를 비활성화 시키는 방법은 응용 프로그램을 너무 많이 신뢰해야 한다는 문제가 있다.

두 번째 단점은 멀티프로세서에서는 적용할 수 가 없다는 것이다.

여러 쓰레드가 여러 CPU 에서 실행 중이라면 각 쓰레드가 동일한 임계 영역을 진입하려고 시도할 수 있다.

이때에 특정 프로세서에서의 인터럽트 비활성화는 다른 프로세서에서 실행 중인 프로그램에는 전혀 영향을 주지 않는다.

결과적으로는 임계 영역에 진입할 수 있다는 말이다.

멀티프로세서가 이제는 매우 흔하게 사용되고 있기 때문에 해결법은 더 정교해야 한다.

세 번째 단점은 장시간 동안 인터럽트를 중지시키는 것은 중요한 인터럽트의 시점을 놓칠 수 있다는 것이다.

때로는 시스템에 심각한 문제를 가져올 수 있다.

예를 들어, CPU 가 저장 장치에서 읽기 요청을 마친 사실을 모르고 지나갔다고 해보자.

운영체제가 이 읽기 결과를 기다리는 프로세스를 언제 깨울 수 있을까?

마지막으로 가장 덜 중요할 수도 있겠지만, 이 방법은 비효율적이다.

일반적인 명령어의 실행에 비해 인터럽트를 비활성화시키는 코드들은 최신의 CPU 들에서는 느리게 실행되는 경향이 있다.

상호 배제를 위해 인터럽트를 비활성화하는 방법은 모든 상황에서 사용하는 것이 아니라, 매우 제한된 경우에만 사용되어야 한다.

예를 들어, 운영체제는 여러 프로세스나 스레드가 동시에 접근하면 문제가 생길 수 있는 내부 자료구조를 안전하게 수정하기 위해, 잠시 동안 인터럽트를 끌 수 있다. 이렇게 하면 해당 코드가 실행되는 동안 다른 작업이 끼어들지 않기 때문에 원자적(atomic)연산처럼 동작할 수 있다.

또한 운영체제는 인터럽트가 자주 발생하면 처리 흐름이 복잡해지거나 예상치 못한 상황이 생길 수 있기 때문에, 특정 구간에서는 인터럽트 발생 자체를 잠시 막기도 한다.

이러한 방식이 가능한 이유는 운영체제 내부에서는 “신뢰할 수 있는 코드”가 실행된다는 전제가 있기 때문이다.

즉, 일반 사용자 프로그램과 달리 운영체제 내부 코드는 시스템이 올바르게 동작하도록 설계된 특권 코드이기 때문에, 이런 방식의 제어가 허용된다.

다만 이런 인터럽트 비활성화는 시스템 전체 반응성을 떨어뜨릴 수 있으므로, 꼭 필요한 짧은 구간에서만 제한적으로 사용되어야 한다.

[6] Test-And-Set(Atomic Exchange)

멀티프로세서에서는 인터럽트를 중지시키는 것이 의미가 없기 때문에, 시스템 설계자들은 락 지원을 위한 하드웨어를 설계하기 시작했다.

1960 년대의 버로우 B5000(Burroughs B500)과 같은 초기의 멀티프로세서 시스템이 하드웨어 지원 기능을 갖고 있다.

오늘날에는 모든 시스템들이 이러한 자원 기능을 갖고 있으며, 단일 CPU 시스템에서도 이런 기능이 존재한다.

하드웨어 기법 중 가장 기본은 Test-And-Set 명령어 또는 원자적 교체(atomic exchange)라고 불리는 기법이다.

Test-And-Set 의 동작을 이해하기 위해서 간단한 플래그 변수로 락을 구현해 보자.

[간단 코드]

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *mutex) {
    // 0 -> 락이 사용 가능함, 1 -> 락 사용중
    mutex->flag = 0;
}

void lock(lock_t *mutex){
    while(mutex->flag == 1) // flag 변수를 검사(TEST) 함
        ;
    mutex->flag = 1; // 이제 설정(SET)
}

void unlock(lock_t *mutex);
    mutex->flag = 0;
}
```

위 코드의 아이디어는 간단하다.

간단한 변수를 사용하여 쓰레드가 락을 획득하였는지를 나타낸다.

임계 영역에 진입하는 첫 쓰레드가 lock()을 호출하여 플래그 값이 1 인지

검사(test)하고 (첫 번째 경우는 1 이 아니다),

플래그의 값을 1 로 설정(set)하여 이 쓰레드가 락을 보유(hold)하고 있다고 표시한다.

임계 영역에서 나오면 쓰레드가 unlock()을 호출하여 플래그 값을 초기화하여 락을 더 이상 보유하고 있지 않다고 표시한다.

만약 첫 번째 쓰레드가 임계 영역 내에 있을 때 다른 쓰레드가 lock()을 호출하면, 그 쓰레드는 while 문으로 spin-wait 을 하며 처음 쓰레드가 unlock()을 호출하여 플래그를 초기화하기를 기다리란.

처음 쓰레드가 플래그를 초기화하면 대기하던 쓰레드는 while 문에서 빠져나와 플래그를 1 로 설정하고 임계 영역 내로 진입한다.

이 코드에는 두 가지 문제가 있다.

하나는 정확성이고 다른 하나는 성능이다.

정확성의 문제는 병행 프로그래밍에 익숙해지면 쉽게 알아 볼 수 있을 것이다.

위 코드가 동작 중이라고 가정하자.

시작할 때 flag = 0 이라고 하자.

아주 타이밍이 정확하게 맞아서 (운 좋게/나쁘게) 인터럽트가 발생하면 두 쓰레드 모두 플래그를 1 로 설정하는 경우가 생길 수 있어서 임계 영역에 두 쓰레드 다 진입할 수 있게 된다.

이러한 현상을 전문가들은 “잘못 만들었다.”고 한다.

상호 배제 제공이라는 기본 요구 조건 보장에 실패하였기 때문이다.

성능의 문제는 이후에 다시 다루겠지만, 사용 중인 락을 대기하는 방법에 문제가 있다.

spin-wait 라는 방법을 사용하여 플래그의 값을 무한히 검사하는데, 이 방법은 다른 쓰레드가 락을 해제할 때까지 시간을 낭비한다.

이 방법은 단일 프로세서에서 특히 매우 손해가 크다.

락을 소유한 쓰레드조차 실행할 수 없기(적어도 문맥 교환 전까지는) 때문이다.

하나씩 정교한 기법을 만들면서 이런 낭비를 없애는 바업을 다루게 될 것이다.

[7] 진짜 돌아가는 스핀 락의 구현

스핀 락은 여러 스레드가 동시에 임계 영역에 들어가지 못하도록 하기 위한 가장 기본적인 락 방식이다.

이 방식의 핵심은 Test-and-Set 이라는 원자적(atomic) 명령어를 사용하는 것이다. 이 명령어는 메모리 값을 읽는 동작(test)과 새로운 값을 저장하는 동작(set)을 하나의 연산으로 동시에 수행한다는 특징이 있다.

예를 들어, 락 변수 flag 가 0 이면 락이 비어 있는 상태이고, 1 이면 다른 스레드가 이미 락을 사용 중인 상태라고 하자.

어떤 스레드가 Test-and-Set 을 호출하면 현재 flag 의 값을 반환하면서 동시에 flag 를 1 로 설정한다.

만약 이전 값이 0 이었다면 해당 스레드는 락을 획득하게 되고, 1 이었으면 이미 다른 스레드가 사용 중이므로 계속 실패하면서 반복하게 된다.

이렇게 하나의 명령어 안에서 검사와 설정이 동시에 이루어지기 때문에 여러 스레드가 동시에 접근하더라도 오직 하나의 스레드만 락을 획득할 수 있다.

이와 같이 Test-and-Set 을 기반으로 구현된 스핀 락은 락이 풀릴 때까지 while 문 안에서 계속 반복하면서 기다리는 구조를 가진다.

이 때문에 CPU 를 계속 사용하면서 대기한다는 특징이 있으며, 이를 스핀(spin)이라고 한다.

```

int TestAndSet(int *old_ptr, int new) {
    int old = *old_ptr; // old_ptr 의 이전 값을 가져옴
    *old_ptr = new;     // old_ptr 에 'new'의 값을 저장함
    return old;         // old 의 값을 반환함
}

```

```

typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    lock->flag = 0;
}

void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        ; // spin
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}

```

한편 단순히 구현한 락도 존재하는데, 이는 while 문에서 flag 값을 검사한 뒤, 반복이 끝나면 flag 를 1로 설정하는 방식이다.

이 방식은 겉보기에는 스핀 락과 비슷하게 while 을 사용하지만 본질적으로 큰 차이가 있다.

단순 구현 방식에서는 flag 를 검사하는 동작과 값을 설정하는 동작이 분리되어 있기 때문에 두 동작 사이에 다른 스레드가 끼어들 수 있다.

예를 들어, 두 스레드가 동시에 flag 가 0임을 확인한 뒤 각각 flag 를 1로 설정하면, 둘 다 임계 영역에 진입하는 문제가 발생할 수 있다.

즉, 이 방식은 원자성이 보장되지 않기 때문에 상호 배제를 제대로 만족하지 못한다.

반면, Test-and-Set 기반 스핀 락은 검사와 설정이 하나의 원자적 연산으로 수행되기 때문에, 이러한 경쟁 조건이 발생하지 않는다.

따라서 while 문은 단순한 대기가 아니라 “원자적 락 획득 시도”자체를 반복하는 구조가 된다.

결과적으로 두 방식 모두 while 을 사용하지만, 단순 락은 “검사 후 설정”구조라 위험하고, Test-and-Set 방식은 “검사와 설정이 동시에 이루어지는 원자적 연산”이라는 점에서 근본적인 차이가 있다.

[8] 스핀 락 평가

스핀 락은 가장 기본적인 형태의 락으로, 그 성능과 특성을 세 가지 기준-상호 배제의 정확성, 공정성, 성능-으로 평가할 수 있다.

먼저 상호 배제 측면에서 보면, 스핀 락은 올바르게 동작한다.

Test-and-Set 과 같은 원자적 연산을 사용하기 때문에 한 번에 하나의 스레드만 락을 획득할 수 있으며, 이를 통해 임계 영역에 동시에 여러 스레드가 들어가는 문제를 방지할 수 있다.

다음으로 공정성 측면을 보면 스핀 락은 좋은 특성을 보이지 않는다.

스핀 락은 어떤 스레드가 먼저 락을 요청했는지와 관계없이, 락이 풀리는 순간 경쟁에서 이긴 스레드만 임계 영역에 들어갈 수 있다.

따라서 대기 중인 스레드들이 순서대로 기회를 얻는다는 보장이 없으며, 어떤 스레드는 계속해서 락을 획득하지 못하고 무한히 기다리는 “기아 상태(starvation)”에 빠질 수도 있다.

마지막으로 성능 측면에서는 시스템 환경에 따라 차이가 크게 나타난다.

단일 CPU 환경에서는 스핀 락의 성능이 매우 비효율적일 수 있다.

임계 영역에서 실행 중이던 스레드가 선점(preemption)되면, 락을 기다리는 다른 스레드들이 CPU 를 계속 하용하면서 while 문을 반복하게 되는데, 이는 CPU 사이클을 낭비하는 결과를 초래한다.

특히 스케줄러가 여러 대기 스레드들을 계속 깨우고 다시 실행시키는 상황이 반복되면 전체 시스템 효율이 크게 떨어진다.

반면 다중 CPU 환경에서는 스핀 락이 상대적으로 더 효율적으로 동작할 수 있다.

예를 들어, CPU 1에서는 스레드 A가 락을 보유하고 실행 중이고, CPU2에서는 스레드 B가 락을 얻기 위해 대기하면서 while 루프를 돌고 있다고 가정하자.

이 경우 스레드 A가 임계 영역을 빠르기 종료하면 곧바로 락이 해제되고, 스레드 B는 비교적 짧은 시간 안에 락을 획득할 수 있다.

또한, 다른 CPU에서 대기하는 동안에도 컨텍스트 스위치 없이 단순히 회전하며 기다리기 때문에, 임계 영역이 짧다면 오히려 비효율적일 수 있다.

결과적으로 스핀 락은 상호 배제는 정확하게 보장하지만 공정성은 보장하지 못하며, 성능은 단일 CPU에서는 비효율적이고 다중 CPU 환경에서는 조건에 따라 효율적으로 동작할 수 있는 락이라고 평가할 수 있다.

[9] Compare-And-Swap

Compare-And-Swap(CAS)은 또 다른 하드웨어 수준의 원자적 동기화 명령어로, SPARC 이나 x86 같은 시스템에서 지원된다.

이 명령어의 핵심 아이디어는 메모리에 저장된 값이 우리가 예상한 값과 같은지를 먼저 비교한 뒤, 같을 경우에만 새로운 값으로 바꾸는 것이다.

```
int CompareAndSwap(int *ptr, int expected, int new) {  
    int actual = *ptr;  
    if(actual == expected)  
        *ptr = new;  
    return actual;  
}
```

구체적으로 보면 CompareAndSwap 은 세 가지 값을 받는다:

- 메모리 주소(ptr)
- 기대 값(expected)
- 새로운 값(new)

먼저, ptr 이 가리키는 실제 값(actual)을 읽고, 이 값이 expected 와 같으리 비교한다. 만약 두 값이 같다면 ptr 에 new 값을 저장하고, 다르면 아무것도 변경하지 않는다. 그리고 항상 원래 메모리 값을 반환한다.

이 방식으로 인해 여러 스레드가 동시에 락을 시도하더라도, 오직 “기대한 값이 그대로 유지된 경우에”만 하나의 스레드가 성공적으로 값을 변경할 수 있다.

이를 이용하면 Test-and-Set 과 동일하게 스핀 락을 구현할 수 있으며, 락이 비어 있는 상태(0)일 때만 1로 바뀌면서 락을 획득하게 된다.

이미 다른 스레드가 락을 가지고 있다면 계속 실패하고 while 문 안에서 반복하게 된다.

```
void lock(lock_t *lock) {  
    while(CompareAndSwap(&lock->flag, 0, 1) != 1)  
        ; // 회전  
}
```


따라서 겉으로 보면 CAS 기반 스핀 락도 Test-and-Set 과 매우 비슷하게 동작한다. 둘 다 원자적 연산을 이용해 락을 얻고, 실패하면 계속 반복(spin)하면서 기다리는 구조를 가진다.

하지만 CAS 는 단순히 값을 설정하는 것뿐만 아니라 “현재 값이 내가 예상한 상태인지”를 먼저 확인한다는 점에서 더 일반적이고 강력한 연산이다. 이 특성 덕분에 CAS 는 단순 락뿐만 아니라 다양한 동기화 구조(예:lock-free 자료구조)를 만드는 데도 사용될 수 있다.

결과적으로 CAS 기반 락은 Test-and-Set 기반 스핀 락과 기본적인 동작 방식은 유사하지만, 더 표현력이 높고 확장성이 큰 원자적 연산이라는 점에서 차이가 있다.

[10] Load-Linked 그리고 Store-Conditional

Load-Linked(LL)와 Store-Conditional(SC)는 Compare-And-Swap(CAS)와 비슷하게 원자적 동기화를 제공하는 하드웨어 명령어 쌍이다.

이 방식의 핵심은 “특정 메모리 값을 읽어둔 뒤, 그 값이 중간에 변하지 않을 때만 쓰기를 허용한다”는 점에 있다.

구체적으로 Load-Linked 는 특정 메모리 주소의 값을 읽어오는 역할을 한다. 이때 단순히 값을 읽는 것이 아니라, 이후에 그 값이 변경되었는지를 추적할 수 있는 상태를 함께 설정한다.

이후 Store-Conditional 은 해당 주소에 새로운 값을 저장하려고 시도하지만, LL 이후 다른 스레드가 그 값을 변경했다면 저장은 실패하게 된다.

즉, SC 는 성공하거나 실패하는 “조건부 저장” 연산이며, 성공했을 때만 실제로 메모리가 업데이트 된다.

```
int LoadLinked(int *ptr){
    return *ptr;
}

int StoreConditional(int *ptr, int value) {
    if (no one has updated *ptr since LoadLinked) {
        *ptr = value;
        return 1;
    } else {
        return 0;
    }
}
```

이러한 동작 방식 덕분에 여러 스레드가 동시에 동일한 변수를 수정하려고 해도, 오직 LL로 읽은 값이 그대로 유지된 경우에만 SC가 성공하게 된다. 이를 이용하면 Test-and-Set과 유사하게 락을 구현할 수 있으며, 예를 들어 flag 값이 0(락이 비어 있음)일 때, 이를 1로 바꾸는 방식으로 락을 획득하게 된다. 만약 다른 스레드가 먼저 값을 변경했다면 SC는 실패하고, 해당 스레드는 다시 시도해야 한다.

결론으로 보면 LL/SC 기반 락은 Test-and-Set이나 CAS 기반 스핀락과 매우 비슷하게 동작한다. 모두 원자적으로 사용해 락을 획득하고, 실패하면 while 루프에서 계속 반복하며 기다리는 스핀 구조를 가진다.

하지만 중요한 차이는 LL/SC가 “읽은 값이 이후에도 그대로 유지되었는지”를 하드웨어 수준에서 직접 검사한다는 점이다. CAS가 “현재 값 == 기대 값 인지 비교 후 변경”하는 방식이라면, LL/SC는 “처음 읽은 이후 변화가 없으면 그때만 저장”이라는 시간 흐름 기반의 조건을 가진다.

이 때문에 LL/SC는 단순 락 구현뿐만 아니라 더 복잡한 비동기 자료구조나 lock-free 알고리즘을 구성하는 데도 사용될 수 있는 더 일반적인 원자적 동기화 방식이다.

결론적으로 LL/SC 기반 락은 Test-and-Set 기반 스핀락과 겉보기에는 비슷하지만, 값의 변경 여부를 추적하는 방식이 더 정교하고 확정성이 높다는 점에서 차이를 가진다.

```
void lock(lock_t *lock) {
    while (1) {
        while (LoadLinked(&lock->flag) == 1)
            ; // 계속 대기 (busy waiting)

        if (StoreConditional(&lock->flag, 1) == 1)
            return; // 락 획득 성공
    }
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

```

void lock(lock_t *lock) {
    while (LoadLinked(&lock->flag) ||
           !StoreConditional(&lock->flag, 1))
        ;
}

```

[11] Fetch-And-Add

Fetch-And-Add 는 메모리 값을 원자적으로 증가시키면서 이전 값을 반환하는 하드웨어 연산이다.

즉, 어떤 변수를 읽고 1 을 더하는 과정이 끊기지 않고 한 번에 수행되며, 동시에 여러 스레드가 접근해도 서로 충돌하지 않는다.

```

int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}

```

이 연산을 이용하면 티켓 락(ticket lock)이라는 공정한 락을 구현할 수 있다.

티켓 락은 하나의 변수만 사용하는 기존 방식과 달리, ticket 과 turn 이라는 두 개의 변수를 이용한다.

ticket 은 “락을 요청한 순번” turn 은 “현재 락을 사용할 수 있는 순번”을 의미한다.

락을 획득하려는 스레드는 Fetch-And-Add 를 통해 ticket 값을 하나씩 증가시키고, 그 반환값을 자신의 순번(myturn)으로 저장한다.

이후 스레드는 turn 값이 자신의 myturn 과 같아질 때까지 기다린다.

즉, “내 차례가 올 때까지 스핀하면서 대기하는 구조”이다.

현재 turn 값이 myturn 과 같아지면 해당 스레드는 임계 영역에 진입할 수 있다.

락을 해제할 때는 turn 값을 증가시켜 다음 순번의 스레드에게 차례를 넘긴다.

이렇게 하면 ticket 을 받은 순서대로 정확하게 임계 영역에 진입하게 된다.

이 방식의 가장 중요한 특징은 “완전한 공정성”이다.

이전의 Test-and-Set 이나 CAS 기반 스핀락에서는 어떤 스레드는 계속 경쟁에서 밀려 반복적으로 실패할 수 있는 반면, 티켓 락에서는 한 번 ticket 을 받은 스레드는 반드시 자신의 순서가 보장된다.

즉, 스레드들은 모두 줄을 서서 순차적으로 실행되는 구조가 된다.

결과적으로, Fetch-And-Add 기반 티켓 락은 기존 스핀락과 동일하게 원자적 연산과 busy waiting 을 사용하지만, 공정성을 보장하여 starvation 문제를 해결한 락 방식이라는 점에서 중요한 차이를 가진다.

```
typedef struct __lock_t {
    int ticket;
    int turn;
} lock_t;

void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn = 0;
}

void lock(lock_t *lock) {
    int myturn = FetchAndAdd(lock->ticket);
    while(lock->turn != myturn)
        ; //회전
}

void unlock(lock_t *lock) {
    FetchAndAdd(&lock->turn);
}
```

[12] 요약

방식	말투	핵심 아이디어	특징	문제
TAS (Test-and-Set)	“비었으면 내가 바로 먹는다”	무조건 1로 바꾸고 확인	가장 단순한 spinlock	CPU 계속 낭비, cache 폭발
CAS (Compare-and-Swap)	“값이 맞으면 바꿔준다”	old 값 기준 조건부 변경	lock-free 구조 가능	실패 시 retry 반복
TTAS (Test-and-Test-and-Set)	“먼저 보고, 비었을 때만 달려간다”	read → write 순서 최적화	TAS보다 cache 효율 좋음	여전히 spin
LL/SC (Load-Linked / Store-Conditional)	“내가 본 뒤 안 바뀌었으면 바꿔줘”	변경 감지 기반 원자성	CAS 대안 (ARM 등)	실패 시 재시도
Fetch-and-Add	“번호표 주고 1 증가”	원자적 증가 + 이전 값 반환	counter/queue 기반	단독 락 아님
Ticket Lock	“번호표 뽑고 순서대로 들어간다”	FIFO spinlock	완전 공정 (starvation 없음)	turn 대기 spin, cache bouncing

[12-1] 한 줄 감각 정리

TAS: 일단 내가 먹는다(무지성 경쟁)

CAS: 조건 맞으면 바꾼다(정밀제어)

TTAS: 보고 들어간다(현실 최적화)

LL/SC: 내가 본 상태 그대로면 성공

Fetch-and-Add: 번호표 뽑기 시스템

Ticket Lock: 번호표 순서대로 입장

[경쟁 기반]

TAS → CAS → TTAS



[감지 기반]

LL/SC



[순서 기반]

Fetch-and-Add → Ticket Lock

[1-4] 병행성: 락 기반의 병행 자료 구조

핵심 질문: 자료 구조에 락을 추가하는 방법

- 특정 자료 구조가 주어졌을 때, 어떤 방식으로 락을 추가해야 그 자료 구조가 정확하게 동작하게 만들 수 있을까?
- 더 나아가, 자료 구조에 락을 추가하여 여러 스레드가 그 자료 구조를 동시에 사용이 가능하도록 하면(병행성) 고성능을 얻기 위해 무엇을 해야 할까?

다른 주제로 넘어가기전에 먼저 흔하게 사용되는 자료 구조에서 락을 사용하는 방법을 살펴보자.

자료구조에 락을 추가하여 사용할 수 있도록 만들면 그 구조는 스레드 사용에 안전(스레드 안전, thread safe)하다고 할 수 있다.

물론, 락이 어떤 방식으로 추가 되었느냐에 따라 그 자료 구조의 정확성과 성능을 좌우할 것이다.

모든 자료 구조를 다 다루거나 병행성이 가능하도록 하는 모든 기법을 다루기는 힘들다.

수년동안 연구되어 온 주제라서 수천 편의 논문들이 이 주제를 다루고 있기 때문이다.

여기서는 어떤 사고 방식으로 이 주제를 접근해야 하는지를 소개하고자 한다.

의문점이 생겼을 때 참고할 수 있는 좋은 자료들을 소개하고자 한다.

여기에 나온 대부분의 정보는 Moir 와 Shavit 의 연구를 참고하였다.

[1] 병행 카운터

동기화되지 않은 카운터는 매우 단순하게 구현할 수 있지만, 여러 스레드가 동시에 접근할 경우 값 증가 연산이 서로 겹치면서 정확성이 깨지는 문제가 발생한다. 이를 해결하기 위해 가장 기본적인 방법은 뮤텝스(lock)를 사용하는 임계 영역을 보호하는 것이다.

락 기반 병행 카운터에서는 `value++` 연산을 수행하기 전에 락을 획득하고, 작업이 끝난 후에 락을 해제한다.

이렇게 하면 동시에 하나의 스레드만 카운터 값을 수정할 수 있으므로 정확한 결과를 보장할 수 있다.

즉, 모든 연산을 직렬화(serialization)하여 경쟁 조건을 제거하는 방식이다.

이 방식은 개념적으로 매우 간단하고 안전하지만, 확장성 측면에서는 큰 문제가 있다. 여러 스레드가 동시에 증가 연산을 수행하려고 하면 결국 하나의 락을 두고 경쟁하게 되며, 많은 스레드가 락을 얻기 위해 대기하면서 성능이 급격히 저하된다. 실제로 스레드 수가 증가할수록 이상적으로는 처리량이 증가해야 하지만, 락 경쟁으로 인해 오히려 전체 수행 시간이 크게 늘어나는 현상이 발생한다.

이러한 문제를 해결하기 위해 등장한 것이 “확장 가능한 카운터(scalable counter)” 개념이다.

그 대표적인 예가 슬로피 카운터(sloppy counter)로, 하나의 전역 카운터만 사용하는 것이 아니라 CPU 코어 마다 별도의 지역 카운터를 두는 구조를 가진다.

각 스레드는 먼저 자신의 CPU에 해당하는 지역 카운터를 증가시키며, 이 과정은 지역 락으로 보호되기 때문에 서로 다른 CPU 간에는 경쟁이 발생하지 않는다.

따라서 카운터 증가 연산이 분산되어 훨씬 높은 병렬성을 얻을 수 있다.

이후 일정 횟수(S 값)에 도달하면 지역 카운터의 값을 전역 카운터에 반영하고 다시 초기화한다.

이 방식의 핵심은 “정확성을 약간 희생하고 성능을 크게 얻는 구조”라는 점이다.

S 값이 작으면 전역 카운터와 거의 일치하는 정확한 값을 유지할 수 있지만, 성능은 낮아지고, S 값이 크면 성능은 크게 향상되지만 전역 카운터 값은 실제 값과 일정 수준 차이가 발생하게 된다.

결과적으로 병행 카운터 설계는 단순히 락으로 보호하는 방식에서 시작하지만, 성능 문제를 해결하기 위해 지역 분산과 근사 업데이트 방식을 사용하는 구조로 발전하게 된다.

이는 정확성과 확정성 사이의 균형을 어떻게 설계할 것인가를 병행 자료구조의 핵심 문제임을 보여준다.

락이 없는 카운터

```
typedef struct {
    int value;
} counter_t;

void init(counter_t *c) {
    c->value = 0;
}

void increment(counter_t *c) {
    c->value++;
}

void decrement(counter_t *c) {
    c->value--;
}

int get(counter_t *c) {
    return c->value;
}
```


락을 사용한 쓰레드 안전 카운터

이를 해결하기 위해 락을 추가하면 다음과 같이 구현할 수 있다.

```
#include <pthread.h>

typedef struct {
    int value;
    pthread_mutex_t lock;
} counter_t;

void init(counter_t *c) {
    c->value = 0;
    pthread_mutex_init(&c->lock, NULL);
}

void increment(counter_t *c) {
    pthread_mutex_lock(&c->lock);
    c->value++;
    pthread_mutex_unlock(&c->lock);
}

void decrement(counter_t *c) {
    pthread_mutex_lock(&c->lock);
    c->value--;
    pthread_mutex_unlock(&c->lock);
}

int get(counter_t *c) {
    pthread_mutex_lock(&c->lock);
    int rc = c->value;
    pthread_mutex_unlock(&c->lock);
    return rc;
}
```

이 방식은 모든 연산에서 락을 사용하여 임계 영역을 보호한다.

즉, 한 번에 하나의 쓰레드만 value 를 수정할 수 있도록 보장한다.

이러한 구조는 모니터(monitor) 방식과 유사하다.

- 함수 호출 시 자동으로 락을 획득하고, 끝나면 해제하는 구조

문제: 성능(확장성)

이 카운터는 정확하게 동작하지만 성능 문제가 있다.

- 모든 연산이 하나의 락을 공유
- 스레드가 많아질수록 락 경쟁 증가
- 결국 성능이 급격히 저하

실험 결과

- 1 개 스레드: 약 0.03 초
- 2 개 스레드: 5 초 이상(오히려 느려짐)
- 이유: 스레드가 늘어나도 동시에 실행되지 못하고 락 때문에 기다리기 때문

이상적인 목표: 완벽한 확장성

이상적으로는

- CPU 가 많아질수록 작업 처리 속도가 함께 증가해야 한다.

이를 완벽한 확장성(Perfect Scaling)이라고 한다.

의미

- 스레드 수 증가 → 성능도 비례해서 증가
- 작업을 병렬로 처리하여 실행 시간이 늘어나지 않음

확장성 있는 카운팅(Sloppy Counter)

앞에서 본 락 기반 카운터는 정확하게 동작한다는 장점이 있지만, 큰 문제가 하나 있다.

바로 쓰레드가 많아질수록 성능이 급격히 떨어진다는 것이다.

왜 이런 일이 발생할까?

모든 쓰레드가 하나의 공유 변수(value)를 수정하려고 할 때, 반드시 하나의 락을 획득해야 하기 때문이다.

즉, 여러 CPU가 있어도 동시에 작업하지 못하고, 결국 줄 서서 기다리는 구조가 된다.

이 문제를 해결하기 위해 나온 방법이 엉성한 카운터(sloppy counter)이다.

핵심 아이디어: 나눠서 처리하자

엉성한 카운터의 핵심은 단순하다.

하나의 카운터를 여러 개로 나눈다.

- 전역 카운터(Global Counter) → 전체 값 저장
- 지역 카운터(Local Counter) → CPU마다 하나씩 존재
- 각각의 락 존재(지역 락 + 전역 락)

예: CPU가 4개라면

- 지역 카운터 4개 + 전역 카운터 1개

동작 방식

기존 방식에서는 모든 쓰레드가 전역 카운터를 바로 수정했다.
하지만 영성한 카운터에서는 다르게 동작한다.

1. 먼저 지역에 증가

쓰레드는 전역이 아니라 자신이 속한 CPU 의 지역 카운터만 증가시킨다.

- 해당 지역 카운터에만 락을 걸면 된다.
- 다른 CPU 와 충돌하지 않음

결과: 동시에 여러 쓰레드가 병렬로 증가 가능(빠름)

2. 일정 시점에 전역으로 반영

그럼 전역 카운터는 언제 업데이트될까?

지역 카운터 값이 S 라는 기준값에 도달하면:

- 전역 카운터에 더하고
- 지역 카운터를 0 으로 초기화

S 값의 의미

S 는 “얼마나 자주 전역으로 반영할 것인가”를 결정한다.

예를 들어:

- $S=5 \rightarrow$ 5 번마다 전역 반영
- $S = 1000 \rightarrow$ 1000 번 쌓고 한 번 반영

이 방식은 빠르지만 한 가지 문제가 생긴다.

전역 카운터 값이 항상 정확하지 않다.

왜냐하면 지역 카운터에 아직 반영되지 않은 값이 남아 있기 때문이다.

- 실제 값 = 전역 + (모든 지역 카운터 값)

성능 vs 정확도(핵심 트레이드오프)

이 구조의 핵심은 S 값에 따라 성능과 정확도가 바뀐다는 것이다.

- S 가 작다 $\rightarrow$ 자주 반영 $\rightarrow$ 정확도 증가 / 성능 저하
- S 가 크다 $\rightarrow$ 덜 반영 $\rightarrow$ 정확도 저하 / 성능 증가

성능이 좋아지는 이유는

기존 방식:

모든 쓰레드 → 하나의 락 경쟁 → 병목 발생

영성한 카운터:

각 CPU → 자기 카운터만 사용 → 충돌 거의 없음

즉, CPU가 늘어나면 자연스럽게 성능도 증가

- 확장성(Scalability)이 좋아짐

이 방식이 중요한 이유는 운영체제나 시스템 내부에서는 “완벽한 정확도”보다 “빠른 처리”가 더 중요한 경우가 많기 때문이다.

- 통계 수집

- 요청 개수 카운팅

- 성능 모니터링

- (약간의 오차는 허용 가능)

[2] 병행 연결리스트 - “락은 어디까지 걸어야 할까?”

핵심은 “락을 크게 걸면 안전하지만 느리고, 작게 걸면 빠르지만 복잡하다”는 문제다.

기본적인 병행 연결 리스트는 매우 단순하다.

삽입이나 탐색 같은 연산을 시작하기 전에 락을 걸고, 끝날 때 해제하는 방식이다. 이렇게 하면 여러 스레드가 동시에 리스트를 수정하더라도 데이터가 깨지지 않기 때문에 정확성은 확보된다.

하지만 여기서 중요한 문제가 하나 나온다.

바로 예외 처리다.

예를 들어 malloc()이 실패하면 어떻게 될까?

락을 잡은 상태에서 실패하면 반드시 락을 해제해야 하는데, 이것 놓치면 다른 스레드가 영원히 대기하는 치명적인 버그가 생긴다.

실제로 이런 “예외 경로”에서 버그가 굉장히 많이 발생한다.

그래서 개선된 방법은 구조를 바꾸는 것이다.

핵심 아이디어는

- 메모리 할당 같은 건 락 밖에서 먼저 처리
- 진짜 공유 데이터 수정 부분만 락으로 감싸기

이렇게 하면 락을 잡고 있는 구간이 짧아지고, 예외 처리도 훨씬 안전해진다.

또 하나 중요한 개선은 코드 흐름을 단순하게 만드는 것이다.

예를 들어, 검색 함수에서 중간에 바로 return 하지 않고, break 로 빠져나온 뒤 공통 return 경로를 사용하면

- 락 해제 코드가 한 곳으로 모이고
- “락 안풀고 return 하는 버그”를 줄일 수 있다.

락 + 제어 흐름(return, break, 예외 처리)은 항상 같이 생각해야 한다.

확장성 문제 - “락 하나면 충분하지 않나?”

기본 방식의 가장 큰 문제는 확장성 부족이다.

리스트 전체에 락이 하나만 있으면:

- 한 스레드가 작업 중이면
- 나머지는 전부 기다려야 한다.

그래서 나온 방법이 hand-over-hand locking(lock coupling)이다.

이건 이렇게 동작한다.

- 각 노드마다 락을 둔다.
- 다음 노드 락을 먼저 잡고
- 현재 노드 락을 풀면서 이동한다.

겉으로 보면 좋아 보인다.

- 여러 스레드가 리스트를 동시에 탐색 가능 -> 병행성 증가

하지만 현실은 다르다

- 노드마다 락 잡고 풀고 반복 -> 오버헤드 폭발
- 결과적으로 단일 락보다 느려질 수도 있다.

병행성이 늘어난다고 무조건 성능이 좋아지는 건 아니다.

- 영성한 카운터(Scalable Counter)

```
#include <pthread.h>

#define NUMCPUS 4 // CPU(스레드) 개수 (로컬 카운터 개수)

// 카운터 구조체
typedef struct counter_t {
    int global; // 전체(global) 카운터 값
    pthread_mutex_t glock; // global 값을 보호하는 락

    int local[NUMCPUS]; // 각 CPU(스레드)별 로컬 카운터
    pthread_mutex_t llock[NUMCPUS]; // 각 로컬 카운터를 보호하는 락

    int threshold; // local → global 로 넘기는 기준값
} counter_t;

// 초기화 함수
void init(counter_t *c, int threshold) {
```

```

c->threshold = threshold;    // threshold 설정

c->global = 0;                // global 값 초기화
pthread_mutex_init(&c->glock, NULL); // global 락 초기화

// 각 CPU 별 local 카운터 및 락 초기화
for (int i = 0; i < NUMCPUS; i++) {
    c->local[i] = 0;           // 로컬 카운터 초기화
    pthread_mutex_init(&c->llock[i], NULL); // 로컬 락 초기화
}
}

// 값 증가 함수
void update(counter_t *c, int threadID, int amt) {

    // 해당 스레드의 로컬 락 획득
    pthread_mutex_lock(&c->llock[threadID]);

    // 로컬 카운터 증가
    c->local[threadID] += amt;

    // threshold 를 넘으면 global 로 반영
    if (c->local[threadID] >= c->threshold) {

        // global 락 획득
        pthread_mutex_lock(&c->glock);

        // local 값을 global 에 더함
        c->global += c->local[threadID];

        // global 락 해제
        pthread_mutex_unlock(&c->glock);

        // local 값 초기화
        c->local[threadID] = 0;
    }
}

```



```

    // 로컬 락 해제
    pthread_mutex_unlock(&c->llock[threadID]);
}

// global 값 조회 함수
int get(counter_t *c) {

    // global 값 읽기 위해 락 획득
    pthread_mutex_lock(&c->glock);

    int val = c->global; // global 값 복사

    // 락 해제
    pthread_mutex_unlock(&c->glock);

    // global 값 반환 (local 값은 포함되지 않음)
    return val;
}

```

- 병행 연결 리스트(Concurrent Linked List)

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// 노드 구조
typedef struct node_t {
    int key; // 저장할 값
    struct node_t *next; // 다음 노드를 가리키는 포인터
} node_t;

// 리스트 구조
typedef struct list_t {
    node_t *head; // 리스트의 시작(head)
    pthread_mutex_t lock; // 리스트 전체를 보호하는 락
} list_t;

```

```
// 리스트 초기화
void List_Init(list_t *L) {
    L->head = NULL;                // 처음에는 비어 있음
    pthread_mutex_init(&L->lock, NULL);    // 락 초기화
}
```

```
// 노드 삽입 (맨 앞에 추가)
```

```
int List_Insert(list_t *L, int key) {

    pthread_mutex_lock(&L->lock);    // 리스트 전체 락 획득

    // 새 노드 생성
    node_t *new = (node_t *)malloc(sizeof(node_t));
    if (new == NULL) {
        perror("malloc");
        pthread_mutex_unlock(&L->lock); // 실패 시 락 해제
        return 1; // 실패
    }

    new->key = key;                // 값 설정
    new->next = L->head;           // 현재 head 를 다음으로 연결
    L->head = new;                // 새 노드를 head 로 설정

    pthread_mutex_unlock(&L->lock); // 락 해제

    return 0; // 성공
}
```

```
// 노드 탐색
```

```
int List_Lookup(list_t *L, int key) {

    pthread_mutex_lock(&L->lock);    // 리스트 전체 락 획득

    node_t *curr = L->head;
```

```

// 리스트 순회
while (curr) {
    if (curr->key == key) {
        pthread_mutex_unlock(&L->lock); // 찾으면 바로 락 해제
        return 0; // 성공 (찾음)
    }
    curr = curr->next; // 다음 노드로 이동
}

pthread_mutex_unlock(&L->lock); // 끝까지 못 찾으면 락 해제

return 1; // 실패 (못 찾음)
}

```

- 병행 연결리스트(개선 버전)

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// 노드 구조
typedef struct node_t {
    int key;                // 저장할 값
    struct node_t *next;    // 다음 노드
} node_t;

// 리스트 구조
typedef struct list_t {
    node_t *head;           // 리스트 시작
    pthread_mutex_t lock;   // 리스트 전체 락
} list_t;

// 초기화
void List_Init(list_t *L) {
    L->head = NULL;         // 초기에는 비어 있음
}

```

```

    pthread_mutex_init(&L->lock, NULL);    // 락 초기화
}

// 삽입 (개선된 버전)
void List_Insert(list_t *L, int key) {

    // 락 없이 메모리 할당 먼저 수행 (성능 개선 포인트)
    node_t *new = (node_t *)malloc(sizeof(node_t));
    if (new == NULL) {
        perror("malloc");
        return;
    }

    new->key = key;

    // 실제 공유 데이터 접근할 때만 락 사용
    pthread_mutex_lock(&L->lock);

    new->next = L->head;
    L->head = new;

    pthread_mutex_unlock(&L->lock);
}

// 탐색
int List_Lookup(list_t *L, int key) {
    int rv = 1; // 기본값: 못 찾음

    pthread_mutex_lock(&L->lock);

    node_t *curr = L->head;

    while (curr) {
        if (curr->key == key) {
            rv = 0; // 찾음
            break;
        }
    }
}

```

```

    }
    curr = curr->next;
}

pthread_mutex_unlock(&L->lock);

return rv;
}

```

방식	말투	특징	문제
기존 리스트	“처음부터 끝까지 락 잡는다”	안전	느림
개선 리스트 (이거)	“준비하고 들어간다”	락 시간 감소	여전히 전체 락

[3] 병행 큐

병행 큐는 여러 스레드가 동시에 데이터를 넣고(enqueue) 빼는(dequeue) 작업을 수행할 수 있도록 만든 자료 구조이다.

가장 단순한 방법은 큐 전체를 하나의 락으로 보호하는 것이지만, 이 방식은 삽입과 삭제가 모두 하나의 락에 묶이기 때문에 병렬성이 크게 제한된다.

즉, 한 스레드가 enqueue 를 수행하는 동안 다른 스레드는 dequeue 조차 할 수 없게 된다.

이 문제를 해결하기 위해 Michael 과 Scott 이 제안한 방법은 큐를 보호하는 락을 하나가 아니라 두 개로 나누는 것이다.

하나는 head(삭제 연산용), 다른 하나는 tail(삽입 연산용)에 각각 사용된다.

이 구조의 핵심은 enqueue 와 dequeue 가 서로 다른 락을 사용하도록 분리하는 데 있다.

이렇게 하면 일반적인 상황에서 삽입 연산은 tail 쪽만 건드리고, 삭제 연산은 head 쪽만 건드리기 때문에 두 작업이 서로 간섭하지 않는다.

결과적으로 한 스레드는 데이터를 넣고, 다른 스레드는 동시에 데이터를 꺼낼 수 있어 병행성이 크게 향상된다.

또 하나 중요한 설계 요소는 “더미(dummy) 노드”의 존재이다.

큐를 초기화할 때 실제 데이터가 없는 노드를 하나 미리 만들어 두고, head 와 tail 이 이 노드를 가리키도록 한다.

이 더미 노드는 큐의 시작과 상태를 단순하게 만들어 주고, enqueue 와 dequeue 연산을 깔끔하게 분리할 수 있게 해준다.

즉, 빈 큐인지 아닌지를 처리하는 복잡한 조건을 줄여주는 역할을 한다.

하지만 이 큐 역시 완벽한 것은 아니다.

현재 구조는 단순히 락으로만 동기화를 하고 있기 때문에, 큐가 비어 있거나 가득 찬 경우에 스레드가 “기다리는 기능”은 제공하지 않는다.

예를 들어 dequeue 를 했는데 큐가 비어 있다면 그냥 실패하고 돌아오게 된다.

따라서 실제 멀티스레드 프로그램에서 사용하기 위해서는, 큐가 비어 있을 때는 기다리고, 데이터가 들어오면 깨워주는 메커니즘이 필요하다.

이를 위해 다음 단계에서 등장하는 것이 조건 변수(condition variable)이다.

결과적으로 Michael-Scott 큐는 “하나의 큰 락” 대신 “역할별로 나눈 락”을 사용하여 병행성을 개선한 구조이며, 이후에는 여기에 조건 변수를 결합하여 실제로 usable 한 큐를 완성하게 된다.

- Michael & Scott 병행 큐(두 개의 락)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <assert.h>

// 노드 구조
typedef struct node_t {
    int value;           // 저장할 값
    struct node_t *next; // 다음 노드
} node_t;

// 큐 구조
typedef struct queue_t {
    node_t *head;        // 큐의 앞쪽 (dequeue)
    node_t *tail;        // 큐의 뒤쪽 (enqueue)

    pthread_mutex_t headLock; // head 전용 락
```

```

    pthread_mutex_t tailLock; // tail 전용 락
} queue_t;

// 초기화
void Queue_Init(queue_t *q) {

    // 더미 노드(dummy node) 생성 (빈 큐를 쉽게 처리하기 위함)
    node_t *tmp = (node_t *)malloc(sizeof(node_t));
    tmp->next = NULL;

    q->head = q->tail = tmp; // head 와 tail 이 같은 더미 노드를 가리킴

    pthread_mutex_init(&q->headLock, NULL);
    pthread_mutex_init(&q->tailLock, NULL);
}

// 삽입 (enqueue)
void Queue_Enqueue(queue_t *q, int value) {

    // 새 노드 생성
    node_t *tmp = (node_t *)malloc(sizeof(node_t));
    assert(tmp != NULL);

    tmp->value = value;
    tmp->next = NULL;

    // tail 쪽만 락
    pthread_mutex_lock(&q->tailLock);

    q->tail->next = tmp; // 현재 tail 뒤에 연결
    q->tail = tmp;      // tail 이동

    pthread_mutex_unlock(&q->tailLock);
}

```

```
// 삭제 (dequeue)
int Queue_Dequeue(queue_t *q, int *value) {

    // head 쪽만 락
    pthread_mutex_lock(&q->headLock);

    node_t *tmp = q->head;          // 현재 head (더미 노드)
    node_t *newHead = tmp->next;    // 실제 데이터 노드

    // 큐가 비어있으면
    if (newHead == NULL) {
        pthread_mutex_unlock(&q->headLock);
        return 1; // 실패 (비어 있음)
    }

    *value = newHead->value; // 값 가져오기
    q->head = newHead;      // head 이동

    pthread_mutex_unlock(&q->headLock);

    free(tmp); // 이전 더미 노드 해제

    return 0; // 성공
}
```


[4] 병행 해시 테이블

병행 행시 테이블은 여러 스레드가 동시에 데이터를 삽입하거나 조회할 수 있도록 만든 자료 구조로, 기본적인 아이디어는 해시 함수를 이용해 데이터를 여러 버킷(bucket)으로 나누는데 있다.

각 버킷은 보통 연결리스트로 구현되며, 이 리스트들이 독립적으로 관리된다.

가장 간단한 병행 해시 테이블 구현은 전체 테이블을 하나의 락으로 보호하는 것이지만, 이 방식은 모든 연산이 하나의 락에 무킹기 때문에 병렬성이 크게 제한된다.

즉, 서로 다른 데이터를 다루더라도 하나의 스레드만 작업할 수 있게 되어 성능이 떨어진다.

이를 개선한 방식에서는 각 해시 버킷마다 별도의 락을 두는 구조로 사용한다. 즉, 어떤 키가 특정 버킷에 매핑되면 해당 버킷의 락만 획득하여 작업을 수행한다. 이렇게 하면 서로 다른 버킷에 접근하는 스레드들은 서로 간섭하지 않고 동시에 작업을 수행할 수 있다.

이 구조의 핵심은 “데이터를 나누고, 락도 나눈다”는 것이다.

해시 함수 덕분에 데이터가 여러 버킷으로 분산되기 때문에, 자연스럽게 락 경쟁도 줄어들게 된다.

결과적으로 여러 CPU에서 동시에 연산을 수행할 수 있어 병행성이 크게 향상된다.

실제 성능 측면에서도 이러한 차이는 명확하게 나타난다.

단일 락을 사용하는 연결 리스트 기반 구조는 스레드 수가 증가할수록 락 경쟁이 심해져 확장성이 급격히 떨어진다.

반면, 버킷별 락을 사용하는 해시 테이블은 스레드들이 서로 다른 버킷을 동시에 처리할 수 있기 때문에, CPU 수가 증가할수록 처리량도 함께 증가하는 좋은 확장성을 보인다.

결과적으로 병행 해시 테이블은 “하나의 큰 락” 대신 “잘게 나뉜 여러 락”을 사용하여 성능 개선한 대표적인 예이다.

이는 병행 자료 구조 설계에서 중요한 원칙인 락 분할(lock partitioning) 또는 세분화된 락(granularity) 개념을 잘 보여주며, 병렬성을 높이기 위해서는 데이터를 나누는 것과 동시에 락도 함께 나누는 것이 핵심이라는 점을 보여준다.

- 병행 해시 테이블

```
#include <stdio.h>
#include <stdlib.h>

#define BUCKETS 101    // 버킷 개수 (해시 테이블 크기)

// list_t 는 이전에 만든 "락이 있는 연결 리스트"라고 가정
// 각 리스트는 내부적으로 mutex 를 가지고 있음

typedef struct hash_t {
    list_t lists[BUCKETS]; // 각 버킷마다 하나의 리스트
} hash_t;

// 해시 테이블 초기화
void Hash_Init(hash_t *H) {
    for (int i = 0; i < BUCKETS; i++) {
        List_Init(&H->lists[i]); // 각 버킷 리스트 초기화
    }
}

// 값 삽입
int Hash_Insert(hash_t *H, int key) {

    // key 를 이용해 어느 버킷에 넣을지 결정
    int bucket = key % BUCKETS;

    // 해당 버킷 리스트에 삽입
    return List_Insert(&H->lists[bucket], key);
}

// 값 탐색
int Hash_Lookup(hash_t *H, int key) {

    // 같은 방식으로 버킷 찾기
    int bucket = key % BUCKETS;
```

```
// 해당 리스트에서 탐색
return List_Lookup(&H->lists[bucket], key);
}
```

[5] 요약

이번 장에서는 카운터, 연결 리스트, 큐, 해시 테이블과 같은 다양한 병행 자료 구조를 살펴보면서, 병행성 설계에서 중요한 몇 가지 핵심 원칙을 확인할 수 있었다. 가장 먼저 중요한 점은 락(lock)을 사용할 때 코드의 흐름을 매우 신중하게 다뤄야 한다는 것이다.

특히 return, 예외 처리와 같은 분기에서 락을 제대로 해제하지 않으면 심각한 버그로 이어질 수 있기 때문에, 구조적으로 실수를 줄이는 코드 설계가 필요하다.

두 번째 중요한 교훈은 병행성을 높였다고 해서 반드시 성능이 좋아지는 것은 아니라는 점이다.

예를 들어, 락을 잘게 나누거나 복잡한 구조를 도입하면 이론적으로는 동시에 더 많은 작업을 수행할 수 있지만, 실제로는 락 획득/해제 비용이나 관리 오버헤드 때문에 오히려 성능이 떨어질 수도 있다.

즉, 병행성(concurrency)과 성능(performance)은 항상 비례하지 않는다.

세 번째로는 성능 최적화는 필요할 때만 해야 한다는 점이다. 처음부터 복잡한 최적화를 적용하기보다는, 가장 단순한 방법(예: 하나의 큰 락)으로 올바르게 동작하는 시스템을 만든 뒤, 실제로 성능 문제가 발생했을 때 점진적으로 개선하는 것이 바람직하다.

이를 “미숙한 최적화(premature optimization)를 피하라”는 원칙으로 표현할 수 있다.

이러한 원칙은 실제 운영체제 발전 과정에서도 잘 나타난다.

초기의 많은 시스템들은 하나의 큰 락(Big Kernel Lock)을 사용하여 커널 전체를 보호했으며, 이 방식만으로도 한동안 충분히 잘 동작했다.

하지만 멀티코어 시스템이 보편화되면서 이 구조는 심각한 병목이 되었고, 결국 더 세분화된 락 구조로 발전하게 되었다.

즉, 처음에는 단순히 시작하고, 필요에 따라 점진적으로 병행성을 확장하는 것이 현실적인 접근 방식이다.

결과적으로 병행 자료 구조 설계의 핵심은 정확성 → 단순성 → 필요 시 성능 개선의 순서를 지키는 것이다.

복잡한 최적화나 고급 기법은 그 이후의 문제이며, 실제 성능 병목이 확인된
경우에만 도입하는 것이 가장 올바른 접근이다.

[1-5] 컨디션 변수

지금까지 락을 통해 임계 영역을 안전하게 보호하는 방법을 배웠지만, 실제 병행 프로그램에서는 “단순히 동시에 접근을 막는 것”만으로는 부족한 경우가 많다. 많은 상황에서 스레드는 어떤 작업을 계속 진행하기 전에 특정 조건이 만족될 때까지 기다려야 하는 경우가 발생한다.

예를 들어, 부모 스레드가 자식 스레드의 작업이 끝난 이후에만 다음 작업을 수행하고 싶을 때가 대표적인 사례이다.

가장 단순한 해결 방법은 공유 변수를 두고, 부모 스레드가 그 값을 계속 확인하면서 기다리는 것이다.

즉, `while(done == 0)` 같은 반복문을 통해 조건이 참이 될 때까지 계속 검사하는 방식이다.

이 방법은 구현이 쉽고 직관적이지만, CPU를 계속 사용하면서 아무 일도 하지 않는 `busy wait`(스핀) 상태가 된다는 문제가 있다.

즉, 조건이 아직 만족되지 않았는데도 계속 CPU를 점유하며 낭비하게 된다.

이러한 방식은 비효율적일 뿐만 아니라, 시스템 전체 성능에도 악영향을 줄 수 있다.

특히 스레드 수가 많아질수록 이런 “쓸데없는 반복”은 더 큰 부담이 된다.

따라서 단순히 반복문으로 기다리는 방식 대신, 조건이 만족될 때까지 스레드를 잠재웠다가(`sleep`), 조건이 충족되면 다시 깨우는 방식이 필요하다.

이 지점에서 등장하는 것이 바로 컨디션 변수(`condition variable`)이다.

컨디션 변수는 스레드가 어떤 조건이 참이 될 때까지 CPU를 사용하지 않고 대기하도록 만들고, 다른 스레드가 그 조건을 만족시키면 깨워주는 메커니즘이다.

즉, “계속 확인하는 방식”이 아니라 “이벤트가 발생할 때까지 잠들어 있는 방식”으로 동작한다.

결과적으로 컨디션 변수는 락이 해결하지 못하는 문제, 즉 “언제 실행해야 하는가(조건 동기화)”를 해결해주는 도구이다.

락이 “동시에 접근하지 못하게 하는 것”이라면, 컨디션 변수는 “지금 실행해도 되는지 기다리는 것”을 담당한다고 이해하면 된다.

코드 1: 기본(자식 기다리지 않음)

```
#include <stdio.h>
#include <pthread.h>

void *child(void *arg) {
    printf("child\n");
    return NULL;
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");

    pthread_t c;
    pthread_create(&c, NULL, child, NULL);

    printf("parent: end\n");

    return 0;
}
```

코드 2: busy-wait(공유 변수 사용)

```
#include <stdio.h>
#include <pthread.h>

volatile int done = 0;

void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");

    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
```

```
while (done == 0)
    ; // busy wait

printf("parent: end\n");

return 0;
}
```

[1] 정의와 루틴들

컨디션 변수(condition variable)는 어떤 조건이 참이 될 때까지 스레드를 기다리게 하고, 조건이 만족되면 깨우는 동기화 도구이다.

내부적으로는 “대기 중인 스레드들의 큐”라고 생각하면 이해하기 쉽다.

어떤 스레드는 조건이 아직 만족되지 않으면 이 큐에서 잠들고, 다른 스레드가 상태를 바꾸면 `signal()`을 통해 이들을 깨운다.

컨디션 변수의 핵심 연산은 두 가지이다.

`wait()`는 조건이 만족될 때까지 스레드를 잠재우는 역할을 하고,

`signal()`은 조건이 만족되었음을 알리며 대기 중인 스레드를 깨운다.

여기서 매우 중요한 점은 `wait()`는 반드시 락(mutex)과 함께 사용된다는 것이다.

`wait()`의 동작은 단순히 “잠든다”가 아니라, 다음과 같은 세 단계로 이루어진다.

1. 호출 시점에 가지고 있던 락을 해제하고
2. 스레드를 잠재운 뒤
3. 깨어날 때 다시 락을 획득하고 나서 리턴한다.

이 과정이 중요한 이유는 경쟁 조건(race condition)을 막기 위해서이다.

만약 락을 잡지 않은 상태에서 조건을 검사하고 잠들면, 조건이 바뀌는 타이밍을 놓쳐서 영원히 깨어나지 못하는 문제가 발생할 수 있다.

Condition Variable 기반

```
#include <stdio.h>
#include <pthread.h>

// 공유 변수
int done = 0;

// mutex 와 condition variable 초기화
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

// child 가 종료할 때 호출
void thr_exit() {
    pthread_mutex_lock(&m);

    done = 1;                // 상태 변경
    pthread_cond_signal(&c);  // 기다리는 스레드 깨움

    pthread_mutex_unlock(&m);
}

// child thread
void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

// parent 가 child 기다리는 함수
void thr_join() {
    pthread_mutex_lock(&m);

    while (done == 0) {      // 조건이 만족될 때까지
        pthread_cond_wait(&c, &m);
    }
}
```



```

    pthread_mutex_unlock(&m);
}

// main
int main(int argc, char *argv[]) {
    printf("parent: begin\n");

    pthread_t p;
    pthread_create(&p, NULL, child, NULL);

    thr_join();                // child 종료까지 기다림

    printf("parent: end\n");

    return 0;
}

```

이를 잘 보여주는 예가 join 구현이다.

부모 스레드는 done 이라는 상태 변수를 확인하면서, 값이 0 이면 wait()를 호출해 잠든다.

자식 스레드는 작업이 끝나면 done 을 1 로 바꾸고 signal()을 호출해 부모를 깨운다. 이때 중요한 것은 상태 변수(done), 락, 컨디션 변수 세 가지가 항상 함께 동작해야 한다는 점이다.

특히 done 과 같은 상태 변수가 꼭 필요한 이유는 “signal 이 먼저 발생하는 경우” 때문이다.

만약 상태 변수가 없다면, 자식 스레드가 먼저 signal 을 보내고 부모는 아직 wait()를 호출하지 않았다면, 그 신호는 그냥 사라진다.

이후 부모가 wait()에 들어가며 아무도 깨워주지 못해 영원히 대기하게 된다.

즉, 컨디션 변수는 상태를 저장하지 않기 때문에 반드시 별도의 상태 변수와 함께 사용해야 한다.

또 하나 매우 중요한 점은 조건을 검사할 때 if 가 아니라 while 을 사용해야 한다는 것이다.

이는 스레드가 깨어났다고 해서 반드시 조건이 만족된다는 보장이 없기 때문이다.

다른 스레드가 먼저 락을 획득해서 상태를 바꿔버릴 수도 있고, 또는 “스푸리어스 웨이크업”처럼 이유 없이 깨어나는 경우도 있기 때문에, 깨어난 후에도 다시 조건을 확인해야 한다.

결과적으로 컨디션 변수의 핵심 패턴은 다음과 같다.

- 락을 잡는다. -> 조건을 while 로 검사 -> 만족하지 않으면 wait() -> 깨어나면 다시 검사 -> 작업 수행 -> 락 해제

이 구조가 복잡해 보이지만, 본질은 단 하나다.

“조건을 안전하게 기다리고, 놓치지 않게 깨우기 위해 락 + 상태 변수 + 컨디션 변수를 함께 사용한다.”는 것이다.

[2] 생산자/소비자(유한 버퍼) 문제

문제 개요

생산자/소비자 문제는 Dijkstra 가 처음 제시한 고전적인 동기화 문제다.

여러 생산자 쓰레드가 데이터를 버퍼에 넣고 여러 소비자 쓰레드가 버퍼에서 데이터를 꺼내 쓰는 구조다.

실생활 예시를 들면:

- 웹 서버에서 HTTP 요청을 큐에 넣는 쪽(생산자)과 꺼내서 처리하는 쪽(소비자)
- `grep foo file.txt | wc -1` 같은 파이프 명령에서 `grep`(생산자)과 `wc`(소비자)사이의 커널 버퍼

유한 버퍼는 공유 자원이므로, 경쟁 조건을 막기 위한 동기화가 반드시 필요하다.

기본 구조: put()과 get()

```
int buffer;
int count = 0;

void put(int value) {
    assert(count == 0); // 버퍼가 비어 있어야 함
    count = 1;
    buffer = value;
}

int get() {
    assert(count == 1); // 버퍼가 차 있어야 함
    count = 0;
    return buffer;
}
```

규칙은 단순하다.

- `count == 0` -> 버퍼가 비어 있음 -> 생산자가 데이터를 넣을 수 있음
- `count == 1` -> 버퍼가 차 있음 -> 소비자가 데이터를 꺼낼 수 있음

시도 1: if문 + 컨디션 변수 하나 -> 버그 있음

```
cond_t cond;
mutex_t mutex;

void *producer(void *arg) {
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        if (count == 1)                // 버퍼가 차면
            Pthread_cond_wait(&cond, &mutex); // 대기
        put(i);
        Pthread_cond_signal(&cond);    // 소비자 깨우기
        Pthread_mutex_unlock(&mutex);
    }
}

void *consumer(void *arg) {
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        if (count == 0)                // 버퍼가 비면
            Pthread_cond_wait(&cond, &mutex); // 대기
        int tmp = get();
        Pthread_cond_signal(&cond);    // 생산자 깨우기
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```

어떤 문제가 생기나?

- 소비자가 2 개(Tc1, Tc2), 생산자가 1 개(Tp)인 상황을 생각해보자.

1. Tc1 이 먼저 실행 -> 버퍼가 비어 있어서 대기
2. Tp 가 버퍼에 데이터를 넣고 -> Tc1 을 깨움
3. 그런데 Tc1 이 실행되기 직전에 Tc2 가 끼어들어서 버퍼를 먼저 비워버림
4. 뒤늦게 깨어난 Tc1 이 빈 버퍼에서 get() 호출 -> 오류 발생

원인:

- 시그널은 단순히 “상태가 바뀌었을 수도 있어”라는 힌트일 뿐이다.
- 쓰레드가 실제로 실행되는 시점에는 상태가 이미 바뀌어 있을 수 있다.
- 이를 Mesa semantic 이라고 한다.

시도 2: while 문 + 컨디션 변수 하나 -> 아직 버그 있음

if 를 while 로 바꾸면 깨어난 후 조건을 다시 확인하므로 위의 문제는 해결된다.

```
while(count == 1)
    Pthread_cond_wait(&cond, &mutex);
```

핵심 규칙:

- Mesa semantic 에서는 항상 while 문을 써야 한다.
- 거짓 깨움(spurious wakeup)에서도 대응할 수 있어 안전하다.

그런데 또 다른 문제가 있다.

이번에 소비자 2 개가 모두 대기 중인 상황을 생각해보자.

1. Tc1, Tc2 모두 버퍼가 비어 있어 대기
2. Tp 가 버퍼에 값을 넣고 -> Tc1 을 깨움
3. Tc1 이 버퍼를 비운 뒤 시그널을 보냄
4. 그런데 Tc2 가 깨어남(생산자 Tp 대신)
5. Tc2 는 버퍼가 비어 있으므로 다시 대기 -> Tp, Tc1, Tc2 모두 대기 상태 -> 교착 상태

원인: 컨디션 변수가 하나뿐이라서, 소비자가 생산자 대신 다른 소비자를 깨울 수 있다.

시도 3: while 문 + 컨디션 변수 두 개 -> 올바른 해법

```
cond_t empty, fill; // 컨디션 변수를 2 개로 분리
mutex_t mutex;

void *producer(void *arg) {
    for(i = 0; i < loops; i++){
        Pthread_mutex_lock(&mutex);
        while(count == 1)
            Pthread_cond_wait(&empty, &mutex); // “빈 자리 생길 때까지 대기”
        put(i);
        Pthread_cond_signal(&fill);           // “채워졌다”고 소비자에게 알림
        Pthread_mutex_unlock(&mutex);
    }
}

void *consumer(void *arg){
    for(i = 0; i < loops; i++){
        Pthread_mutex_lock(&mutex);
        while(count == 0)
            Pthread_cond_wait(&fill, &mutex); // “채워질 때까지” 대기
        int tmp = get();
        Pthread_cond_signal(&empty);         // “비었다”고 생산자에게 알림
        Pthread_mutex_unlock(&mutex);
        printf(“%d\n”, tmp);
    }
}
```

주체	대기하는 조건 변수	시그널 보내는 조건 변수
생산자	empty(빈 자리 기다림)	fill(소비자에게 알림)
소비자	fill(데이터 기다림)	empty(생산자에게 알림)

이렇게 하면 소비자가 실수로 다른 소비자를 깨우는 일이 없어진다.

최종 해법: 버퍼를 여러 개로 확장

버퍼를 배열로 만들면 병행성이 높아지고 문맥 교환이 줄어들어 더 효율적이다.

```
int buffer[MAX];
int fill = 0, use = 0, count = 0;

void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX; // 원형 버퍼
    count++;
}

int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    count--;
    return tmp;
}
```

조건만 살짝 바꾼다.

- 생산자: `count == MAX` 이면 대기(버퍼가 꽉 찼을 때)
- 소비자: `count == 0` 이면 대기(버퍼가 비었을 때)

보너스: `broadcast()`가 필요한 경우

메모리 할당기처럼 스레드마다 필요한 조건이 다를 때는 누구를 깨워야 할지 알 수 없다.

이럴 때는 `signal()` 대신 `broadcast()`로 모든 대기 스레드를 깨운 뒤, 각자 조건을 확인하게 한다.

```
Pthread_cond_broadcast(&c); // 모두 깨우기
```

성능상 손해는 있지만, 이를 포함 조건(covering condition)이라 부르며 정확성을 보장하는 안전한 방법이다.

단, 브로드캐스트로 바꿔야만 프로그램이 동작한다 -> 설계에 버그가 있을 가능성이 높다.

핵심 요약

규칙	이유
조건 검사는 항상 while 사용	깨어났을 때 조건이 바뀌어 있을 수 있음(Mesa semantic)
컨디션 변수를 역할에 맞게 분리	잘못된 스레드를 깨우는 것을 방지
버퍼를 크게 만들기	병행성 향상, 문맥 교환 감소
조건이 복잡하면 broadcast() 고려	모든 경우를 포함하는 안전한 대안